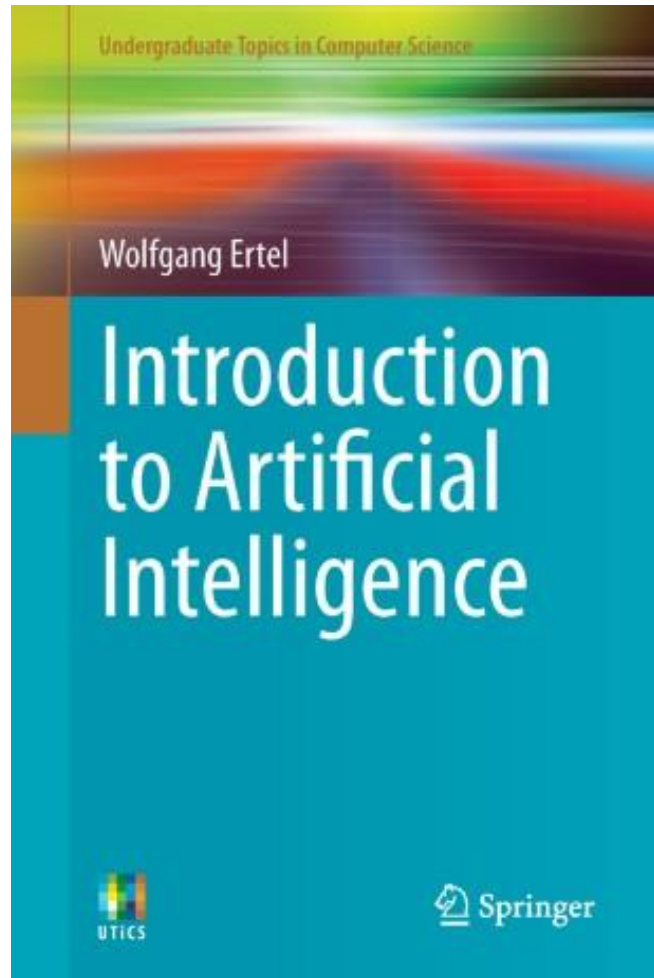


AI Fundamentals

Chapter 3 – Neural Networks



VIVES University of Applied Sciences
Bachelor in electronics-ICT



AI Fundamentals

Chapter 3: Neural Networks

- We would like to thank the following people for their contribution:
 - Stefaan Haspeslagh, VIVES
 - Yves Sagaert, VIVES
 - Andy Louwyck, VIVES
 - Prof. Danny De Schreye, KU Leuven

AI Fundamentals

Chapter 3: Neural Networks

Contents

- 1. Introduction**
2. From Biology to Simulation
3. The Mathematical Model
4. Hopfield Networks
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
7. Support Vector Machines
8. Deep Learning

AI Fundamentals

Chapter 3: Neural Networks

1. Introduction

- Neural networks are networks of **nerve cells** in the **brains** of humans and animals.
- The human brain has about **100 billion nerve cells**.
- We humans owe our intelligence and our ability to learn various motor and intellectual capabilities to the brain's complex relays and adaptivity.
- For many centuries biologists, psychologists, and doctors have tried to understand how the brain functions.
- Around 1900 came the revolutionary realization that these tiny physical building blocks of the brain, the nerve cells and their connections, are responsible for awareness, associations, thoughts, consciousness, and the ability to **learn**.

AI Fundamentals

Chapter 3: Neural Networks

1. Introduction

- The first big step toward neural networks in AI was made **1943** by **McCulloch** and **Pitts** in an article entitled “**A logical calculus of the ideas immanent in nervous activity**” [J. Anderson and E. Rosenfeld. *Neurocomputing: Foundations of Research*. MIT Press, Cambridge, MA, 1988. Collection of fundamental original papers]
- They were the first to present a **mathematical model of the neuron** as the basic switching element of the brain.
- This article laid the foundation for the construction of artificial neural networks and thus for this very important branch of AI.

AI Fundamentals

Chapter 3: Neural Networks

1. Introduction

- We would consider the field of modeling and simulation of neural networks to be the bionics brain within AI.
- Nearly all areas of AI attempt to recreate cognitive processes, such as in **logic** or in **probabilistic reasoning**.
- *However, the tools used for modeling – namely mathematics, programming languages, and digital computers – have very little in common with the human brain.*
- With artificial neural networks, the approach is different.
- Starting from knowledge about the function of natural neural networks, we attempt to **model**, **simulate**, and even **reconstruct** them in **hardware**.

AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
- 2. From Biology to Simulation**
3. The Mathematical Model
4. Hopfield Networks
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
7. Support Vector Machines
8. Deep Learning

2. From Biology to Simulation

- **Figure 3.1** illustrates two stages of modeling of a neural network. Above a **biological model** and below a **formal model** with neurons and directed connections between them.
- Each of the roughly 100 billion neurons in a human brain has, as shown in a simplified representation in **Figure 3.1**, the following structure and function.
- Besides the **cell body**, the neuron has an **axon**, which can make local connections to other neurons over the **dendrites**.
- The **axon** can, however, grow up to a meter long in the form of a nerve fiber through the body.

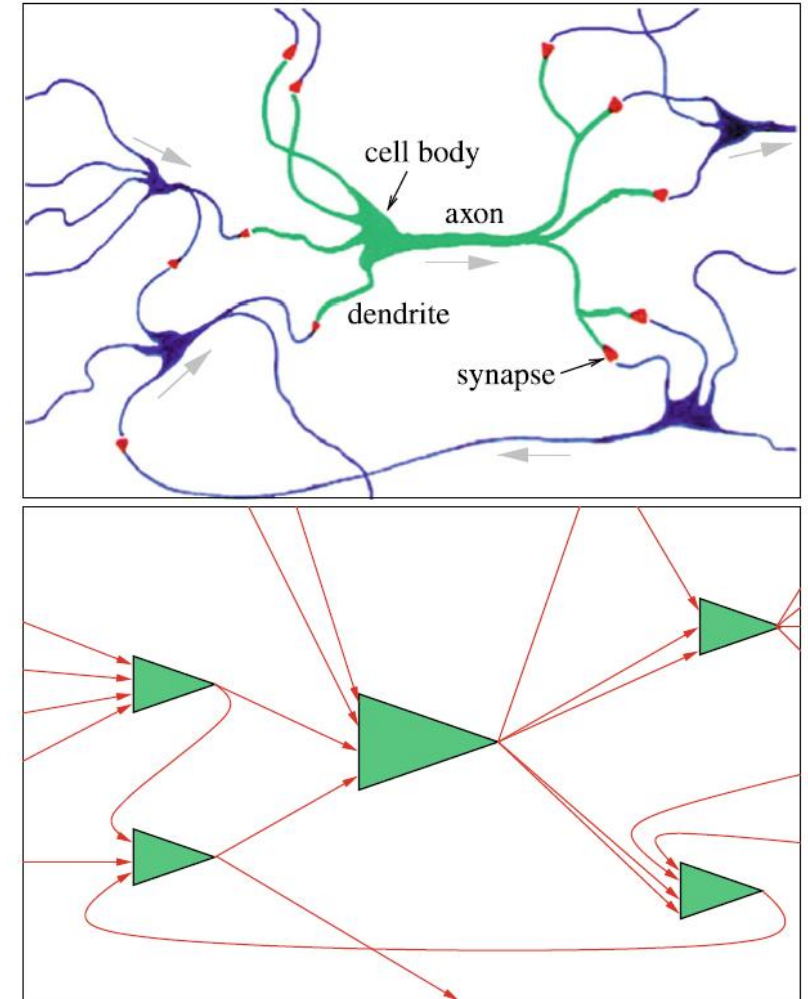


Figure 3.1: Modeling of a neural network

2. From Biology to Simulation

- The **cell body** of the neuron can store small electrical charges, similarly to a *capacitor* or *battery*.
- This storage is loaded by incoming electrical impulses from other neurons.
- The more electric impulse comes in, the higher the voltage. If the voltage exceeds a certain **threshold**, the neuron will fire. This means that it unloads its store, in that it sends a spike over the axon and the synapses.
- The electrical current divides and reaches many other neurons over the synapses, in which the same process takes place.

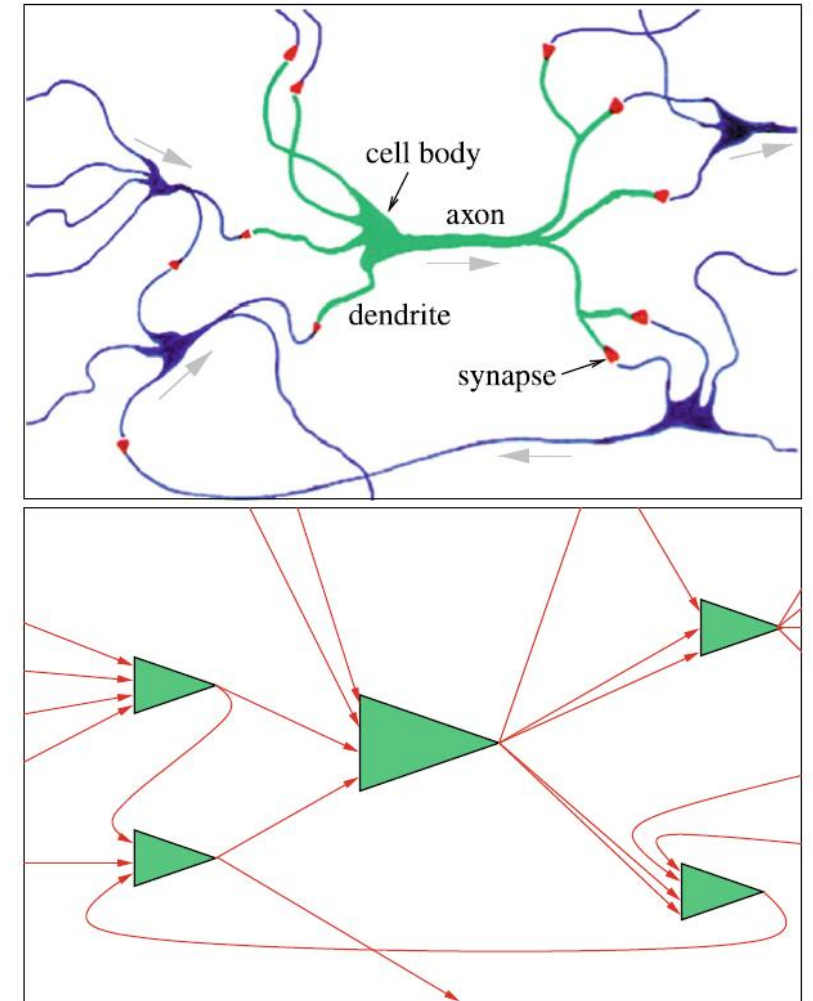


Figure 3.1: Modeling of a neural network

2. From Biology to Simulation

- *Now the question of the **structure** of the neural network arises.*
- Each of the roughly 10^{11} neurons in the brain is connected to roughly 1000 to 10.000 other neurons, which yields a total of over 10^{14} connections.
- If we further consider that this gigantic number of extremely thin connections is made up of soft, three-dimensional tissue and that experiments on human brains are not easy to carry out, then it becomes clear why we do not have a detailed circuit diagram of the brain.
- *Presumably we will never be capable of completely understanding the circuit diagram of our brain, based solely on its immense size.*
- From today's perspective, it is no longer worth even trying to make a complete circuit diagram of the brain, because the structure of the brain is **adaptive**.
- It changes itself on the fly and adapts according to the **individual's activities** and **environmental influences**.

2. From Biology to Simulation

- The central role here is played by the **synapses**, which create the **connection** between neurons.
- At the connection point between two neurons, it is as if two cables meet. However, the two leads are not perfectly conductively connective, rather there is a small **gap**, which the electrons cannot directly jump over.
- This gap is filled with chemical substances, so-called **neuro-transmitters**.
- These can be **ionized** by an applied voltage and then transport a charge over the gap.
- The **conductivity** of this **gap** depends on many parameters, for example the concentration and the chemical composition of the neurotransmitter.
- It is enlightening that the function of the brain reacts very sensitively to changes of this synaptic connection, for example through the influence of **alcohol** or other **drugs**.

AI Fundamentals

Chapter 3: Neural Networks

2. From Biology to Simulation

- *How does **learning** work in such a neural network?*
- The surprising thing here is that it is not the actual active units, namely the **neurons**, which are adaptive, rather it is the **connections** between them, that is, the **synapses**. Specifically, this can change their **conductivity**.
- We know that a synapse is made **stronger** by however much more electrical current it must carry.
- **Stronger** here means that the synapse has a **higher conductivity**.
- **Synapses** which are **used often** obtain an increasingly **higher weight**.
- For synapses which are used *infrequently* or are *not active at all*, the **conductivity** continuous to **decrease**. This can even lead to them *dying off*.

AI Fundamentals

Chapter 3: Neural Networks

2. From Biology to Simulation

- All **neurons** in the brain work **asynchronously** and in **parallel**, but, compared to a computer, at **very low speed**.
- The time for a neural impulse takes about a **millisecond**, exactly the same as the time for the ions to be transported over the synaptic gap.
- The clock frequency of the neuron then is **under 1 kHz** and is thus lower than of modern computers by a factor of 10^6 .
- This disadvantage, however, is more than compensated for in many complex cognitive tasks, such as image recognition, by the **very high degree of parallel processing** in the network of nerve cells.
- The connection to the outside world comes about through **sensor neurons**, for example on the *retina* in the *eyes*, or through *nerve cells* with very long axons which read from the brain to the *muscles* and thus can carry out actions such as the movement of a leg.
- *However, it is still unclear how the principles discussed make intelligent behavior possible.*
- Just like many researchers in neuroscience, we will attempt to explain using simulations of a simple mathematical model how **cognitive tasks**, for example pattern recognition, become possible.

Neural Networks

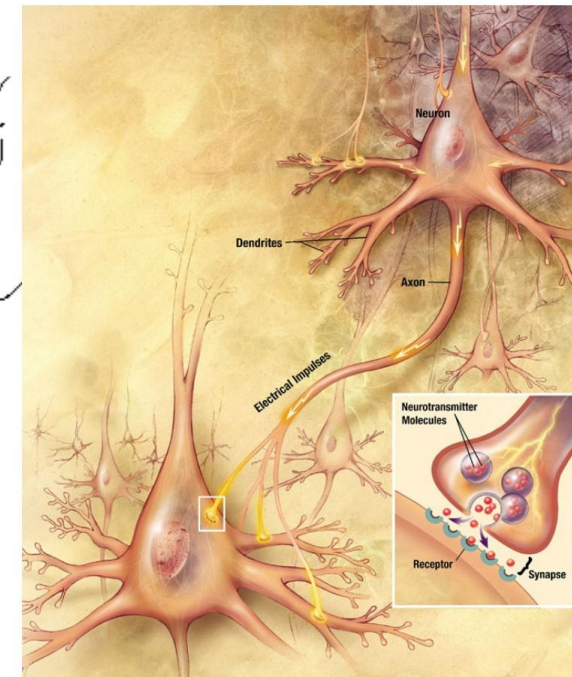
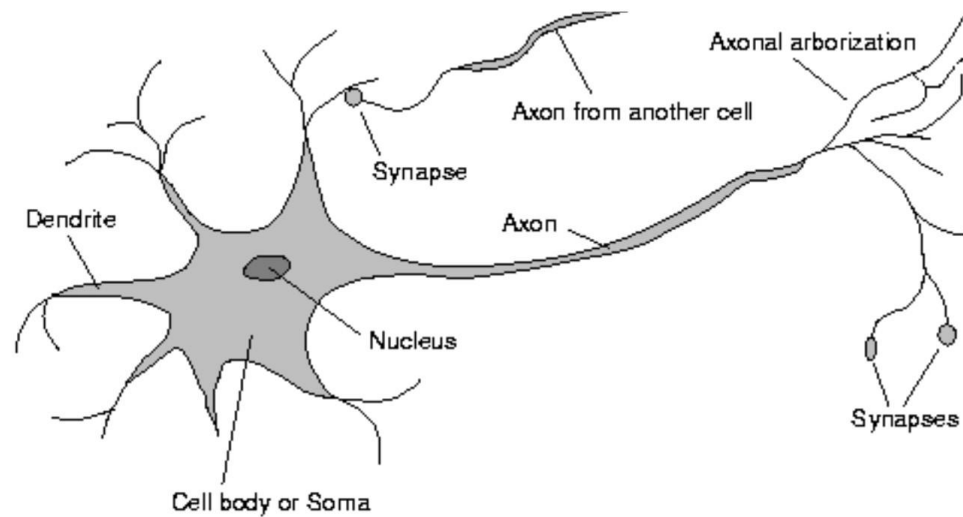
- Origins: Algorithms that try to mimic the brain.
- Very widely used in 80s and early 90s; popularity diminished in late 90s.
- Recent resurgence: State-of-the-art technique for many applications
- Artificial neural networks are not nearly as complex or intricate as the actual brain structure

URL: [Neural Networks](#)

Neural Function

- Brain function (thought) occurs as the result of the firing of **neurons**
- Neurons connect to each other through **synapses**, which propagate **action potential** (electrical impulses) by releasing **neurotransmitters**
 - Synapses can be **excitatory** (potential-increasing) or **inhibitory** (potential-decreasing), and have varying **activation thresholds**
 - Learning occurs as a result of the synapses' **plasticity**: They exhibit long-term changes in connection strength
- There are about 10^{11} neurons and about 10^{14} synapses in the human brain!

Biology of a Neuron



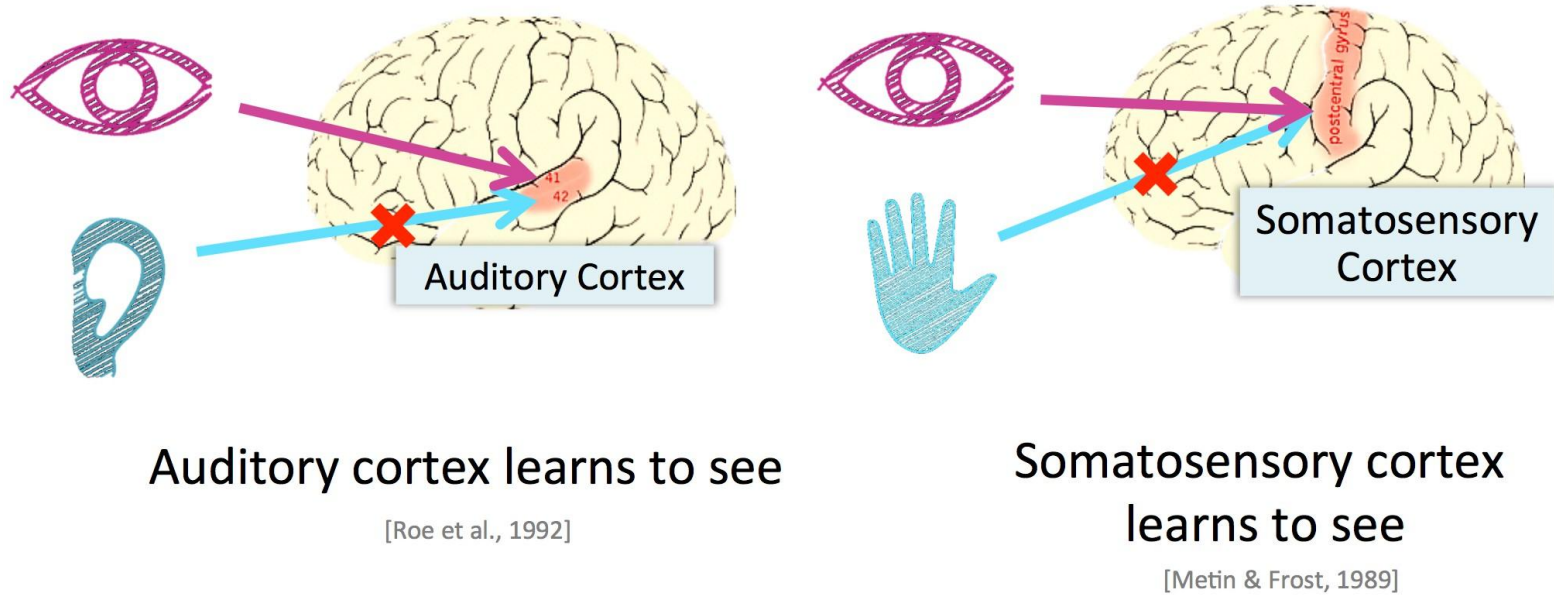
Brain Structure

- Different areas of the brain have different functions
 - Some areas seem to have the same function in all humans (e.g., Broca's region for motor speech); the overall layout is generally consistent
 - Some areas are more plastic, and vary in their function; also, the lower-level structure and function vary greatly
- We don't know how different functions are “assigned” or acquired
 - Partly the result of the physical layout / connection to inputs (sensors) and outputs (effectors)
 - Partly the result of experience (learning)
- We *really* don't understand how this neural structure leads to what we perceive as “consciousness” or “thought”

AI Fundamentals

Chapter 3: Neural Networks

The “One Learning Algorithm” Hypothesis



Comparison of computing power

INFORMATION CIRCA 2012	Computer	Human Brain
Computation Units	10-core Xeon: 10^9 Gates	10^{11} Neurons
Storage Units	10^9 bits RAM, 10^{12} bits disk	10^{11} neurons, 10^{14} synapses
Cycle time	10^{-9} sec	10^{-3} sec
Bandwidth	10^9 bits/sec	10^{14} bits/sec

- Computers are way faster than neurons...
- But there are a lot more neurons than we can reasonably model in modern digital computers, and they all fire in parallel
- Neural networks are designed to be massively parallel
- The brain is effectively a billion times faster

URL: [Difference between brain and computer](#)

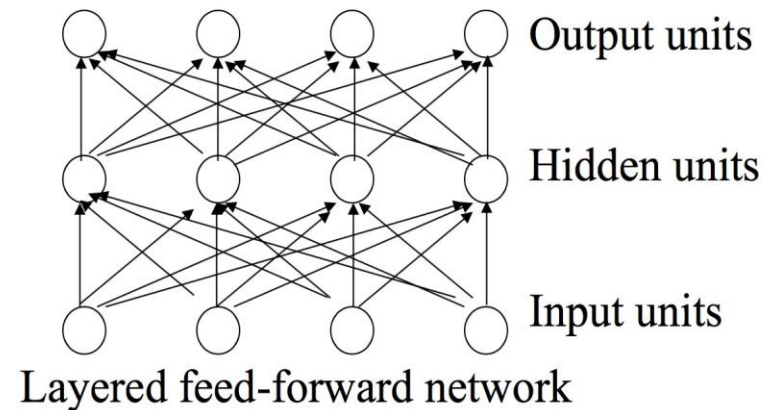
AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
2. From Biology to Simulation
- 3. The Mathematical Model**
4. Hopfield Networks
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
7. Support Vector Machines
8. Deep Learning

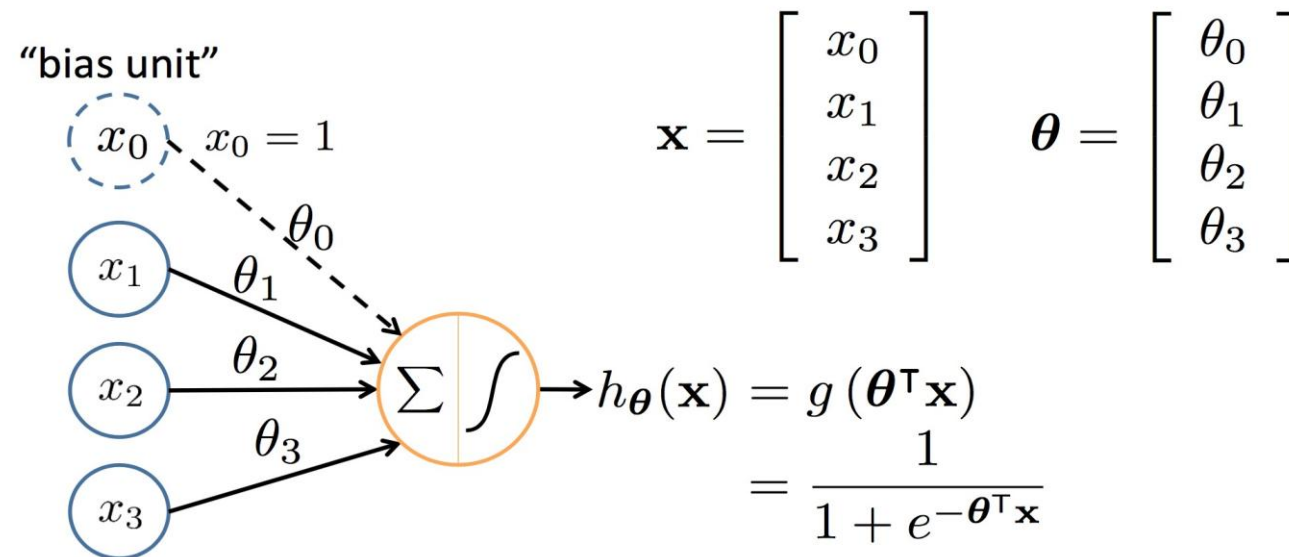
Neural networks



- Neural networks are made up of **nodes** or **units**, connected by **links**
- Each link has an associated **weight** and **activation level**
- Each node has an **input function** (typically summing over weighted inputs), an **activation function**, and an **output**

URL: [Neural networks](#)

Neuron Model: Logistic Unit



Sigmoid (logistic) activation function: $g(z) = \frac{1}{1 + e^{-z}}$

URL: [Neuron Model: Logistic Unit](#)

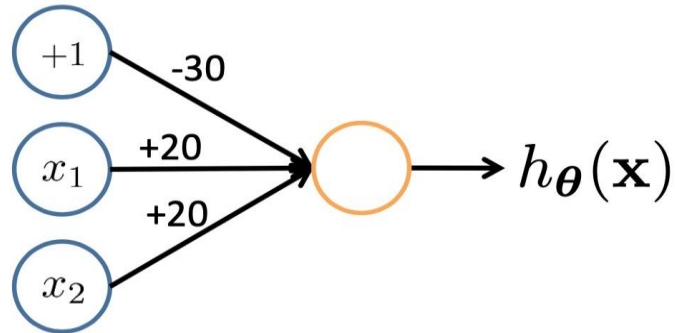
URL: [Activation Functions in Neural Networks](#)

Neural networks: representing Boolean Functions

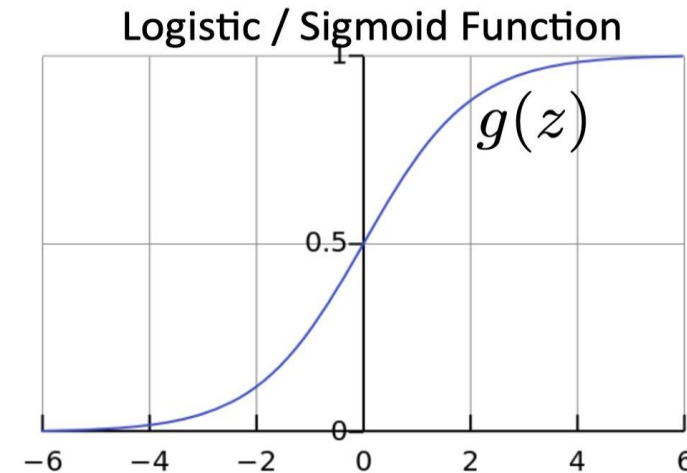
Simple example: AND

$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$



$$h_{\theta}(\mathbf{x}) = g(-30 + 20x_1 + 20x_2)$$

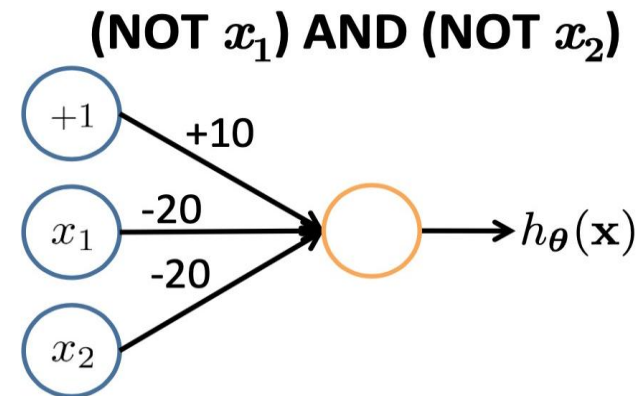
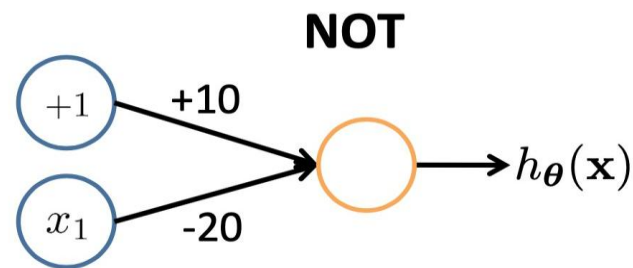
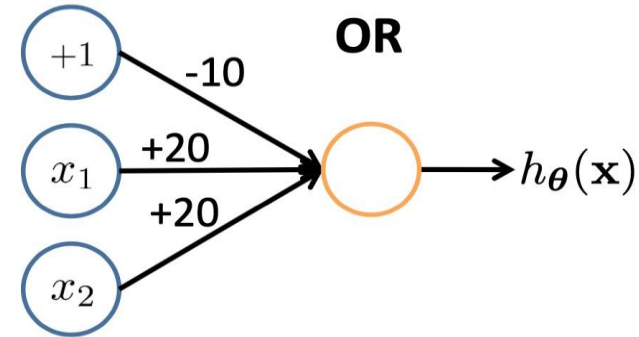
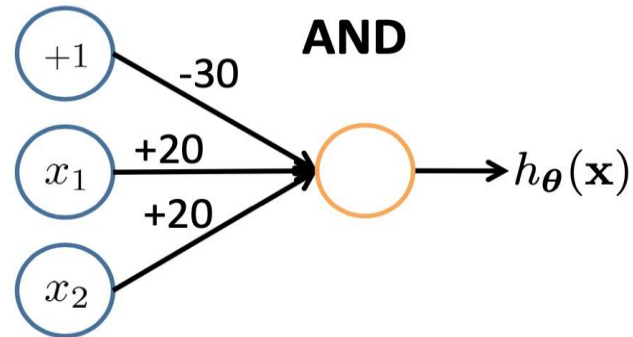


x_1	x_2	$h_{\theta}(\mathbf{x})$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

AI Fundamentals

Chapter 3: Neural Networks

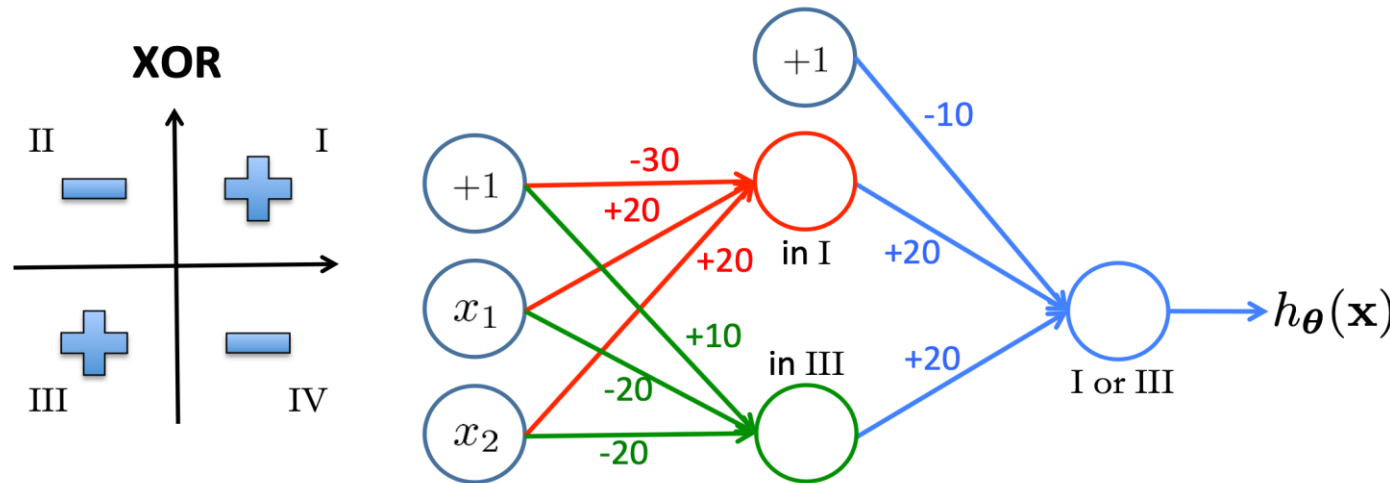
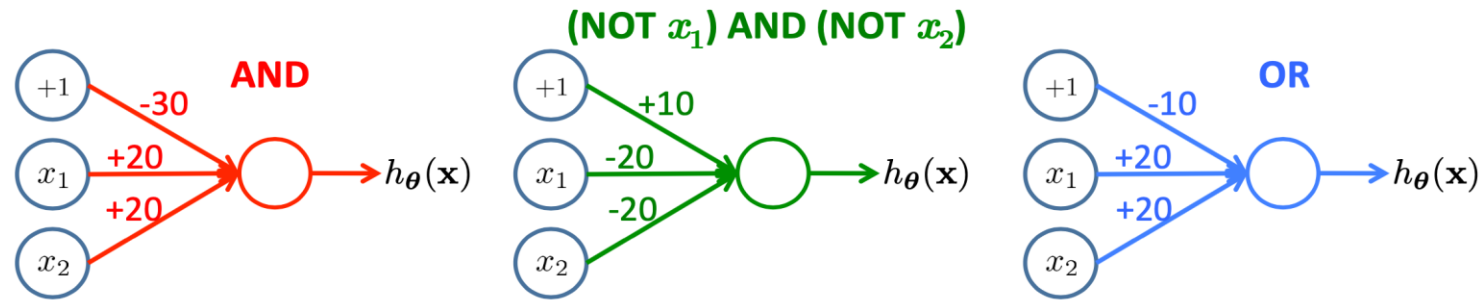
Neural networks: representing Boolean Functions



AI Fundamentals

Chapter 3: Neural Networks

Neural networks: combining representations

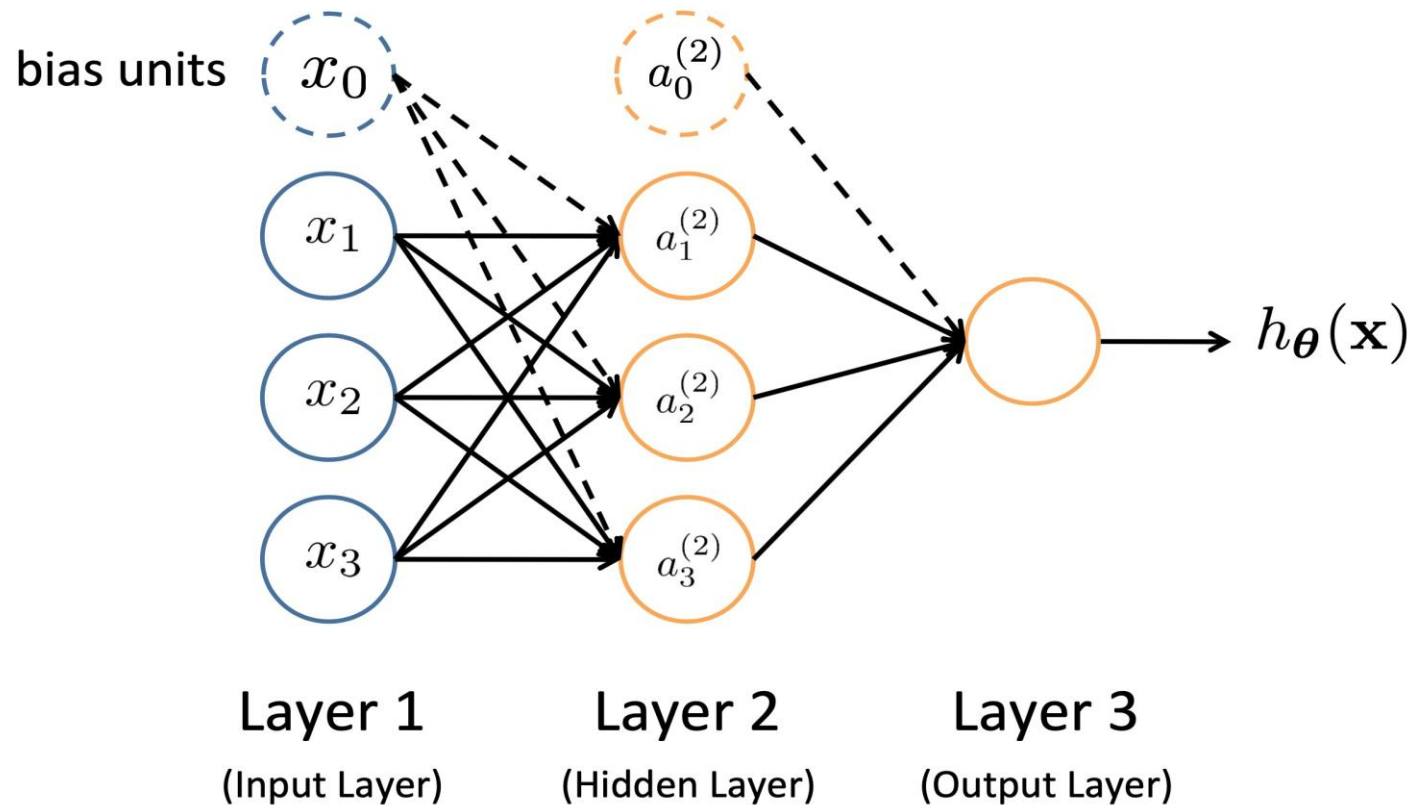


URL: [Combining representations to create non-linear functions](#)

AI Fundamentals

Chapter 3: Neural Networks

Neural networks: networks with multiple layers



AI Fundamentals

Chapter 3: Neural Networks

Neural networks: multiple output units - classification



Pedestrian



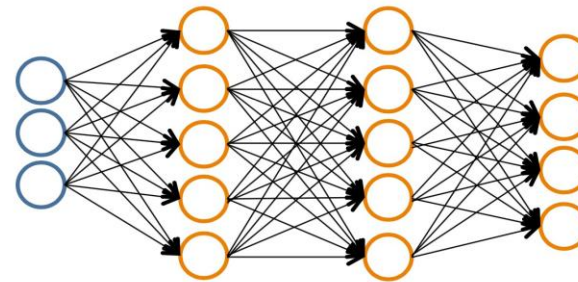
Car



Motorcycle



Truck



$$h_{\Theta}(\mathbf{x}) \in \mathbb{R}^K$$

We want:

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

when pedestrian

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

when car

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

when motorcycle

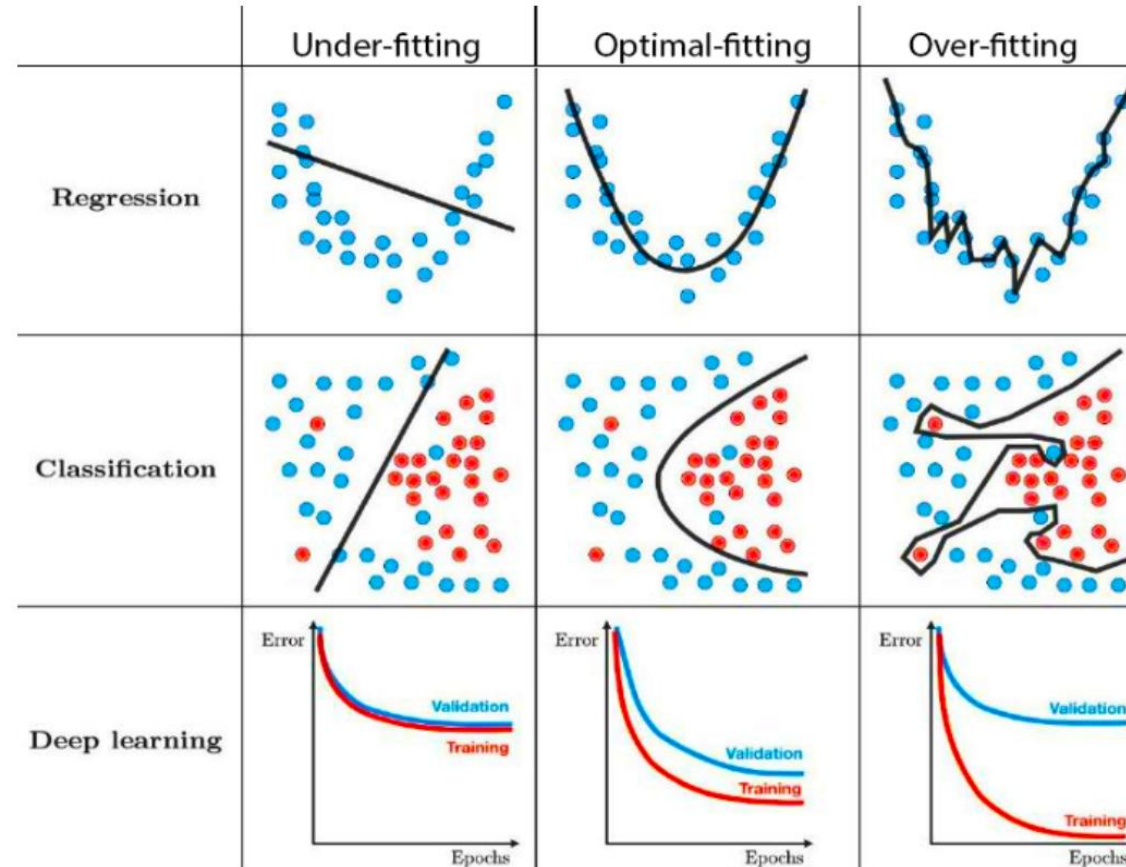
$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

when truck

AI Fundamentals

Chapter 3: Neural Networks

Deep Learning: overfitting still exists



Source: Underfitting, Optimal-fitting and Overfitting for linear regression [1]

3. The Mathematical Model

- First we replace the **continuous time axis** with a **discrete time** scale.
- The neuron i carries out the following calculation in a time step.
- The “loading” of the activation potential is accomplished simply by **summation of the weighted output values** $x_1, \dots, x_n$ of all incoming connections over the formula

$$\sum_{j=1}^n w_{ij} \cdot x_j$$

- This **weighted sum** is calculated by most neural models.
- Then an **activation function** f is applied to it and the result

$$x_i = f \left(\sum_{j=1}^n w_{ij} \cdot x_j \right)$$

- is passed on to the neighboring neurons as output over the synaptic weights.

3. The Mathematical Model

- In **Figure 3.2** this kind of modeled neuron is shown.
- The modeled neuron applies the **activation function** f to the **weighted sum of all inputs**.

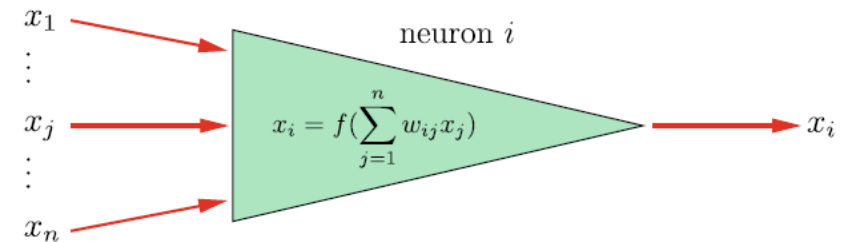


Figure 3.2: The structure of a formal neuron.

- For the activation function there are a number of possibilities.
- The simplest is the **identity**: $f(x) = x$.
- The neuron thus calculates only the weighted sum of the input values and passes this on.
- However, this frequently leads to **convergence problems** with the neural dynamics because the function $f(x) = x$ is **unbounded** and the function values can grow beyond all limits over time.
- Very well restricted, in contrast, is the **threshold function** (**Heaviside step function**)

$$H_{\theta}(x) = \begin{cases} 0 & \text{if } x < \theta, \\ 1 & \text{else.} \end{cases}$$

3. The Mathematical Model

- The whole neuron then computes its output as

$$x_i = \begin{cases} 0 & \text{if } \sum_{j=1}^n w_{ij} \cdot x_j < \theta \\ 1 & \text{else.} \end{cases}$$

- This formula is **identical** to [2.1], in other words, to a **perceptron with the threshold θ** (see Figure 3.3).
- The neuron with a step function works like a perceptron with a threshold.*
- The input neurons 1, ..., n here have only the function of variables which pass on their externally set values $x_1, \dots, x_n$ unchanged.

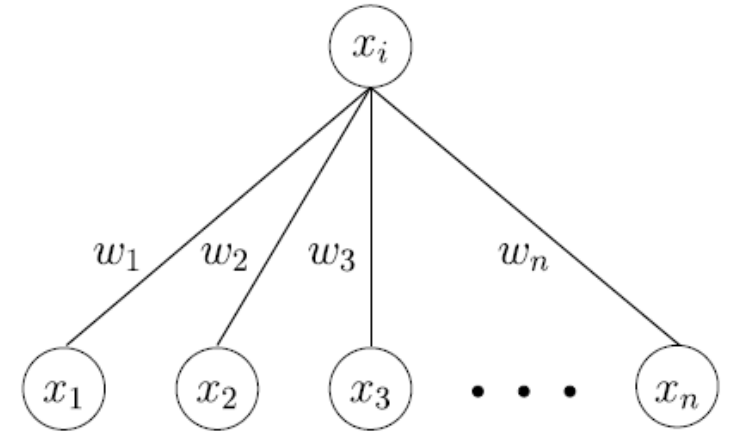


Figure 3.3: The neuron with a step function

3. The Mathematical Model

- The **step function** is quite sensible for **binary neurons** because the activation of a neuron can only take on the values **zero** or **one** anyway.
- In contrast, for **continuous neurons** with activations between 0 and 1, the step function creates a *discontinuity*.
- However, this can be *smoothed out* by a **sigmoid function**, such as

$$f(x) = \frac{1}{1 + e^{-\frac{x-\theta}{T}}}$$

- With the graph in **Figure 3.4**, which illustrates the **sigmoid function** for various values of the parameter **T**. We can see that in the limit $T \rightarrow 0$ the **step function** results.

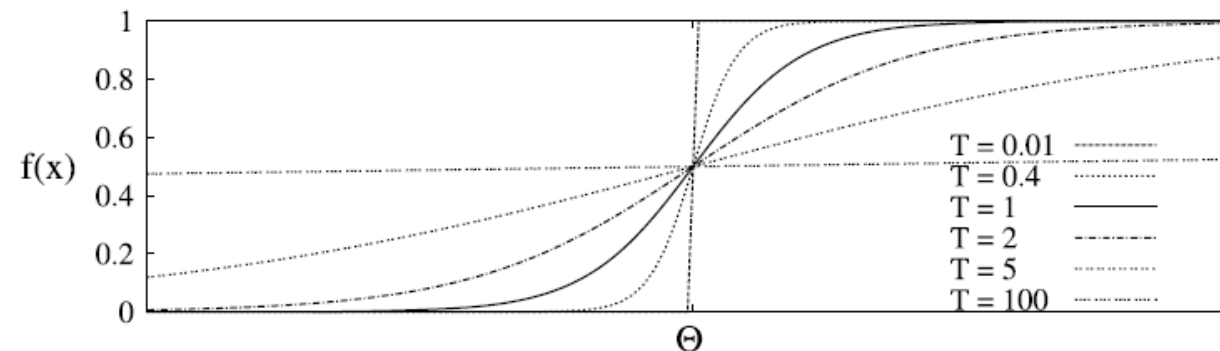


Figure 3.4: The sigmoid function

3. The Mathematical Model

- **Modeling learning** is central to the theory of neural networks.
- As previously mentioned, one possibility of learning consists of strengthening a synapse according to how many electrical impulses it must transmit.
- This principle was postulated by **D. Hebb** in **1949** and is known as **the Hebb rule**:

If there is a connection w_{ij} between neuron j and neuron i and repeated signals are sent from neuron j to neuron i , which results in both neurons being simultaneously active, then the weight w_{ij} is reinforced.

A possible formula for the weight change Δw_{ij} is $\Delta w_{ij} = \eta \cdot x_i \cdot x_j$ with the constant η (learning rate), which determines the size of the individual learning steps.

- There are many modifications of this rule, which then result in different types of networks or learning algorithms.

AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
2. From Biology to Simulation
3. The Mathematical Model
- 4. Hopfield Networks**
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
7. Support Vector Machines
8. Deep Learning

4. Hopfield Networks

- Looking at the **Hebb rule**, we see that for neurons with values between zero and one, the weights can *only grow with time*.
- *It is not possible for a neuron to weaken or even die according to this rule.* This can be modeled, for example, by a **decay constant** which weakens an unused weight by a constant factor per time step, such as 0.99.
- This problem is solved quite differently by the model presented by **Hopfield** in **1982** [*J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. Proc. Natl. Acad. Sci. USA, 79:2254-2558, April 1982*]
- It uses **binary neurons**, but with the two values **-1** for *inactive* and **1** for *active*. Using the **Hebb rule** we obtain a **positive contribution** to the **weight** whenever two neurons are **simultaneously active**.
- If, however, only one of the two neurons is *active*, Δw_{ij} is **negative**.

4. Hopfield Networks

- **Hopfield networks**, which are a beautiful and visualizable example of *auto-associative memory*, are based on this idea.
- **Patterns** can be stored in auto-associative memory.
- To call up a saved pattern, it is sufficient to provide a similar pattern.
- *The store then finds the most similar saved pattern.*

URL: [Hopfield-netwerk](#)

- A classic application of this is **handwriting recognition**.
- In this **learning phase** of a **Hopfield network**, N binary coded patterns, saved in the vectors $q^1, \dots, q^n$, are supposed to be learned.
- Each component $q_i \in \{-1, 1\}$ of such a vector q^j represents a pixel of a pattern.
- For vectors consisting of n pixels, a neural network with n neurons is used, one for each pixel position.
- The neurons are **fully connected** with the restriction that the **weight matrix** is **symmetric** and all **diagonal elements** w_{ij} are **zero**. That is, there is no connection between a neuron and itself.

4. Hopfield Networks

- The **fully connected network** includes complex *feedback loops*, so-called recurrences, in the network (see **Figure 3.5**). **Figure 3.5** illustrates recurrent connections between two neurons in a **Hopfield network**.

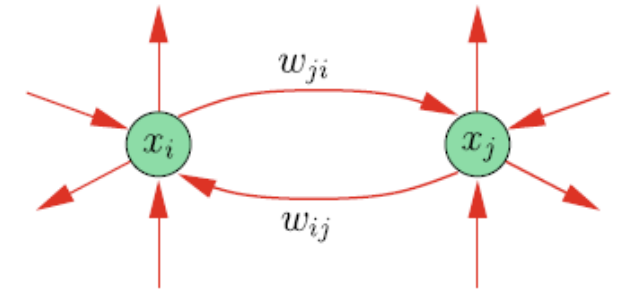


Figure 3.5: Recurrent connections

- N patterns can be learned by simply calculating all weights with the formula

$$w_{ij} = \frac{1}{N} \sum_{k=1}^N q_i^k q_j^k \quad [3.1]$$

- This formula points out an interesting relationship to the **Hebb rule**. Each pattern in which the pixels i and j have the *same* value makes a **positive** contribution to the weight w_{ij} . Each other pattern makes a **negative** contribution.
- Since each pixel corresponds to a neuron, here the weights between the neurons which simultaneously have the **same value**, are being **reinforced**. Please note this small difference to the Hebb rule.

4. Hopfield Networks

- Once all the patterns have been stored, the network can be used for **pattern recognition**.
- We give the network a **new pattern** x and update the activations of all neurons in an asynchronous process according to the rule

$$x_i = \begin{cases} -1 & \text{if } \sum_{j=1}^n \& j \neq i w_{ij} x_j < 0 \\ 1 & \text{else} \end{cases} \quad [3.2]$$

- until the network becomes *stable*, that is, until no more activations change.
- As a program schema this reads as follows:

```
HOPFIELDASSOCIATOR( $q$ )
Initialize all neurons:  $x = q$ 
Repeat
   $i = \text{Random}(1, n)$ 
  Update neuron  $i$  according to (9.2)
Until  $x$  converges
Return ( $x$ )
```


4. Hopfield Networks

Application to a Pattern Recognition Example

- We apply the described algorithm to a simple pattern recognition example.
- It should recognize digits in a 10x10 pixel field.
- The Hopfield network thus has 100 neurons with a total of $\frac{100 \cdot 99}{2} = 4950$ weights.
- For the patterns of the digits 1, 2, 3, 4 in **Figure 3.6** are trained.
- In **Figure 3.6** the dynamics of a **Hopfield network** is illustrated.
- In rows 2, 3 and 4 we can easily see how the network **converges** and the learned pattern is recognized after about 300 to 400 iterations.
- *In the last row several stable states are shown which are reached by the network when the input pattern deviates too much from all learned patterns.*

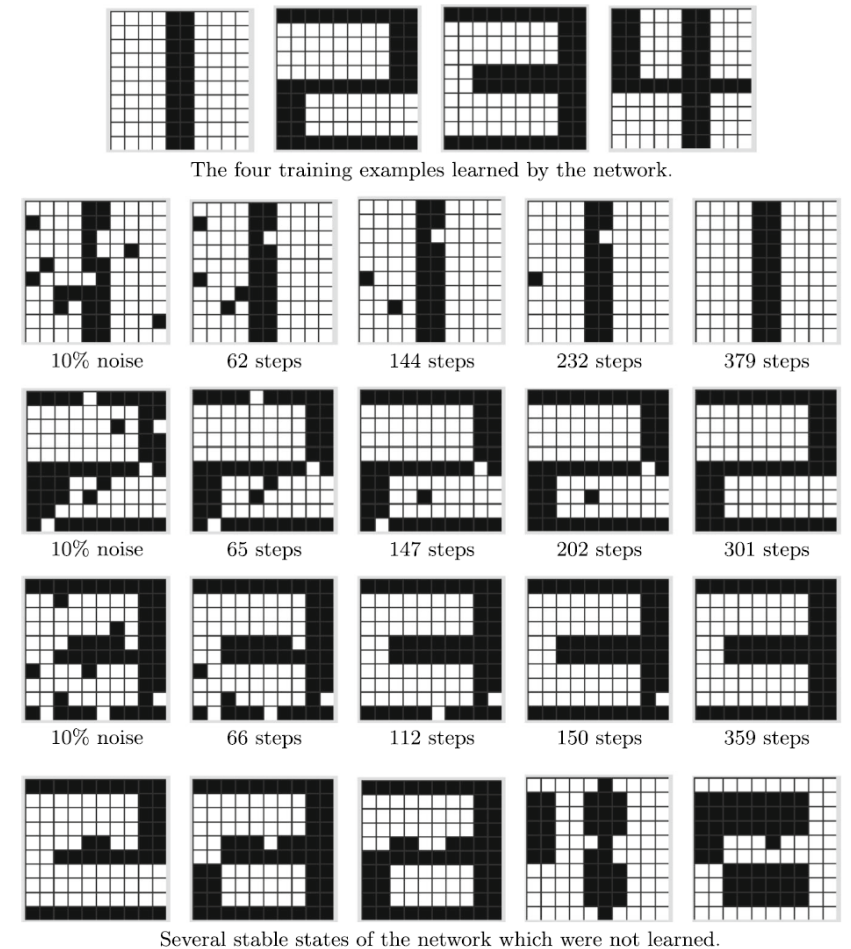
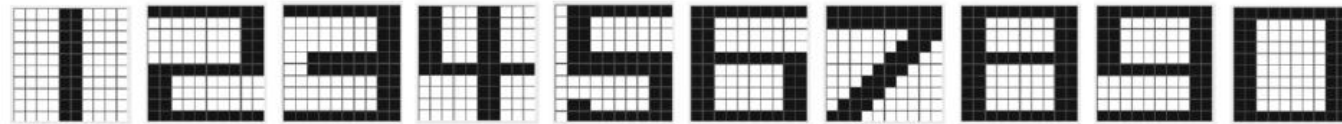


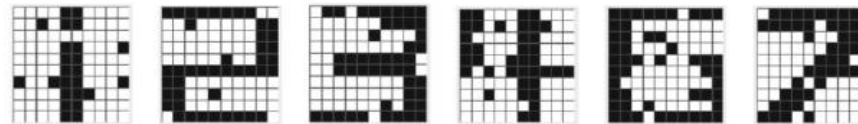
Figure 3.6: Dynamics of a Hopfield network

4. Hopfield Networks - Application to a Pattern Recognition Example

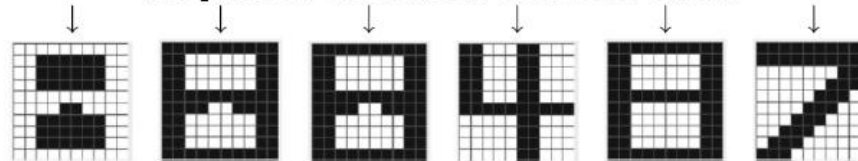
- Now we save the **digits 0 to 9** (Figure 3.7) in the same network and **test** the network again with patterns that have a random amount of about **10% inverted pixels**.
- In the figure, we clearly see that the Hopfield iteration often does *not converge* to the most similar learned state even for only **10% noise**.
- Evidently the network can securely save and recognize four patterns, but for ten patterns its memory capacity is exceeded.



The ten learned patterns.



Six patterns with about 10% noise added.



The stable states that evolved from the noisy patterns.

Figure 3.7: For ten learned states the network shows chaotic behavior

4. Hopfield Networks - Analysis

- In **1982**, **John Hopfield** showed that this model is formally equivalent to a physical model of **magnetism**.
- Small elementary magnets, so-called **spins**, mutually influence each other over their magnetic fields (**Figure 3.8**).

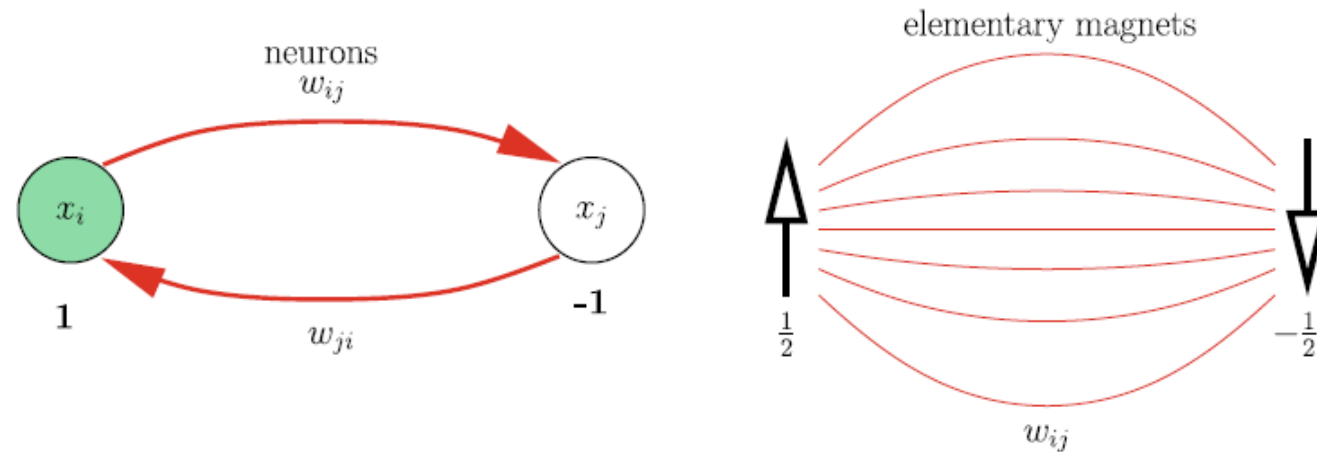


Figure 3.8: Comparison between the neural and physical interpretation of the Hopfield model

- If we observe two such spins i and j , they interact over a constant w_{ij} and the total energy of the system is then
$$E = -\frac{1}{2} \sum_{i,j} w_{ij} x_i x_j$$

4. Hopfield Networks – Analysis

- By the way, $w_{ij} = 0$ in physics too, because particles have no self-interaction, because physical interactions are **symmetric**, $w_{ij} = w_{ji}$
- A physical system in equilibrium takes on a (stable) state of **minimal energy** and thus minimizes $E(x, y)$.
- If such a system is brought into an arbitrary state, then it moves toward a state of **minimal energy**.
- The **Hopfield dynamics** defined in [3.2] correspond exactly to this principle because it updates the state in each interaction such that, of the **two states** **-1** and **1**, the one with *smaller energy* is taken on.
- The contribution of the neuron i to the total energy is $-\frac{1}{2}x_i \sum_{j \neq i}^n w_{ij}x_j$
- If now $\sum_{j \neq i}^n w_{ij}x_j < 0$, then $x_i = -1$ results in a **negative** contribution to the total energy, and $x_i = 1$ results in a **positive** contribution.
- For $x_i = -1$, the network takes on a state of **lower energy** than it does for $x_i = 1$.
- Analogously, we can assert that in the case of $\sum_{j \neq i}^n w_{ij}x_j \geq 0$ it must be true that $x_i = 1$.

4. Hopfield Networks – Analysis

- If each individual iteration of the neural dynamics results in a *reduction of the energy function*, then the total energy of the system *decreases* monotonically with time.
- Because there are only finitely many states, the network moves in time to a state of **minimal energy**.
- *Now we have the exciting question: what do these minima of the energy function mean?*
- As we saw in the **pattern recognition** example, in the case of few learned patterns the system converges to one of the *learned* states. *The **learned patterns** represent **minima** of the **energy function** in the state space.*
- *If however too many patterns are learned, then the system converges to minima which do not correspond to learned patterns. Here we have a transition from an **ordered** dynamics into a **chaotic** one.*
- Hopfield and other physicists have investigated exactly this process and have shown that there is in fact a phase transition at a critical number of learned patterns.
- If the number of learned patterns exceeds this value, then the system changes from the **ordered** phase into the **chaotic**.

AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
2. From Biology to Simulation
3. The Mathematical Model
4. Hopfield Networks
- 5. Linear Networks with minimal errors**
6. The Backpropagation Algorithm
7. Support Vector Machines
8. Deep Learning

AI Fundamentals

Chapter 3: Neural Networks

5. Linear Networks with minimal errors

- The **Hebb rule** used in the neural models presented so far works with **associations** between **neighboring neurons**.
- In associative memory, this is exploited in order to learn a mapping from *query vectors* to *targets*.
- This works very well in many cases, especially when the query vectors are **linearly independent**.
- If this condition is not fulfilled, for example when too much training is available, the question arises: *how do we find the optimal weight matrix?* Optimal means it **minimizes** the **average error**.
- We humans are capable of learning from mistakes. *The Hebb rule does not offer this possibility.*
- The **back propagation algorithm**, described in the following, uses an elegant solution known from function approximation to change the weights such that the **error** on the training data is **minimized**.

5. Linear Networks with minimal errors

- Let N pairs of **training vector**

$$T = \{(q^1, t^1), \dots, (q^N, t^N)\}$$

- be given with $q^p \in [0,1]^n$ $t^p \in [0,1]^m$.
- We are looking for a function $f: [0,1]^n \rightarrow [0,1]^m$ which **minimizes** the **squared error** on the data.

$$\sum_{p=1}^N (f(q^p) - t^p)^2$$

- Let us first assume that the data contains no contradictions. That is, there is no query vector in the training vector which should be mapped to two different targets.
- In this case it is not difficult to find a function that minimizes the squared error.
- In fact, there exist infinitely many functions which make the error zero.*

AI Fundamentals

Chapter 3: Neural Networks

5. Linear Networks with minimal errors

- We define the function

$$f(q) = 0, \quad \text{if } q \notin \{q^1, \dots, q^N\}$$

and

$$f(q^p) = t^p \quad \forall p \in \{1, \dots, N\}$$

- This is a function which even makes the error on the training data **zero**.
- *What more could we want? Why are we not happy with this function?*
- The answer is: because we want to build an **intelligent** system!
- Intelligent means, among other things, that the learned function can **generalize** well from the training data to *new, unknown data* from the same representative data set. In other words it means: we do not want **overfitting** of the data by memorization. *What is it then that we really want?*
- We want a function that is smooth and “evens out” the space between the points.
- **Continuity** and the ability to take **multiple derivatives** would be sensible requirements.
- Because even with these conditions there are still **infinitely** many functions which make the error **zero**, we must restrict this class of functions even further.

5. Linear Networks with minimal errors – Least Squares Method

- The simplest choice is a **linear mapping**.
- We begin with a two-layer network (**Figure 3.9**) in which the single neuron y of the second layer calculates its **activation** using

$$y = f\left(\sum_{i=1}^n w_i x_i\right)$$

- with $f(x) = x$ (**linear** activation function).

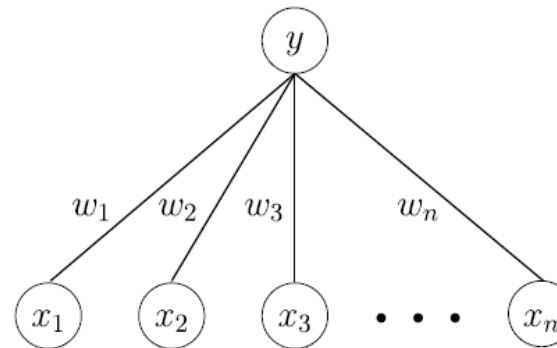


Figure 3.9: A two-layer network with an output neuron

5. Linear Networks with minimal errors – Least Squares Method

- The fact that we are only looking at **output** neurons here does not pose a real restriction because a two-layer network with two or more output neurons can always be **separated** into **independent networks** with identical input neurons for each of the original output neurons.
- The **weights** of the subnetworks are all **independent**.
- Using a **sigmoid function** instead of the **linear activation** does not give any advantage here because the sigmoid function is strictly monotonically increasing and does not change the order between various output values.
- Desired is a **vector** w which minimizes the **squared error**

$$E(w) = \sum_{p=1}^N (wq^p - t^p)^2 = \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right)^2$$

AI Fundamentals

Chapter 3: Neural Networks

5. Linear Networks with minimal errors – Least Squares Method

- As a necessary condition for a **minimum** of this error function **all partial derivatives** must be **zero**.
- Thus we require that for $j = 1, \dots, n$:

$$\frac{\partial E}{\partial w_j} = 2 \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right) q_j^p = 0$$

- Multiplying this out yields

$$\sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p q_j^p - t^p q_j^p \right) = 0$$

- and exchanging the sums results in the **linear system of equations**

$$\sum_{i=1}^n w_i \sum_{p=1}^N q_i^p q_j^p = \sum_{p=1}^N t^p q_j^p$$

- can be written as the matrix equation $\mathbf{A} \cdot \mathbf{w} = \mathbf{b}$
- URL: [Partial derivative](#)
- URL: [Gradient descent](#)

5. Linear Networks with minimal errors – Least Squares Method

- These so-called normal equations always have *at least one solution* and, when A is invertible, exactly **one**.
- Furthermore, the matrix A is positive-definite, which has the implication that the discovered solution in the unique case is a **global minimum**. This algorithm is known as **least squares method**.
- The calculation time for setting up the matrix A grows with $\mathcal{O}(N \cdot n^2)$ and the time for solving the system of equations as $\mathcal{O}(n^3)$.
- This method can be extended very simply to incorporate **multiple output** neurons because, as already mentioned, for two-layer feedforward networks the **output neurons** are *independent* of each other.

5. Linear Networks with minimal errors – The Delta Rule

- **Least squares** is, like the *perceptron* and *decision tree learning*, a so-called **batch learning algorithm**, as opposed to **incremental learning**.
- *In **batch learning**, all training data must be learned in one run.*
- If new training data is added, it cannot simply be learned in addition to what is already there. *The whole learning process must be **repeated** with the enlarged set.*
- This problem is solved by **incremental learning** algorithms, which can **adapt** the learned model to each additional new example.
- In the algorithms we will look at in the following discussion, we will *additively update the weights for each new training example* by the rule

$$w_j = w_j + \Delta w_j$$

5. Linear Networks with minimal errors – The Delta Rule

- To derive an **incremental** variant of the **least squares method**, we reconsider the above calculated n partial derivatives of the error function

$$\frac{\partial E}{\partial w_j} = 2 \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right) q_j^p$$

- The **gradient** $\nabla E = \left(\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right)$ as a vector of all partial derivatives of the error function points in *the direction of the strongest rise of the error function* in the n -dimensional space of the weights.
- While searching for a **minimum**, we will therefore follow the direction of the **negative gradient**.
- As a formula for changing the weights we obtain

$$\Delta w_j = -\frac{\eta}{2} \frac{\partial E}{\partial w_j} = -\eta \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right) q_j^p$$

- where the **learning rate** η is a freely selectable **positive constant**.
- A larger η speeds up convergence but at the same time raises the risk of *oscillation around minima*.

5. Linear Networks with minimal errors – The Delta Rule

- Therefore, the optimal choice of η is *not a simple task* (see **Figure 3.10**).
- A large η , for example $\eta = 1$, is often used to start with, and then *slowly shrunk*.
- **Figure 3.10** illustrates the **gradient descent** for a large η (left) and a very small η (right) into a valley descending flatly to the right.
- For a large η there are **oscillations** around the valley.
- For a η which is too small, in contrast, convergence in the flat valley happens *very slowly*.

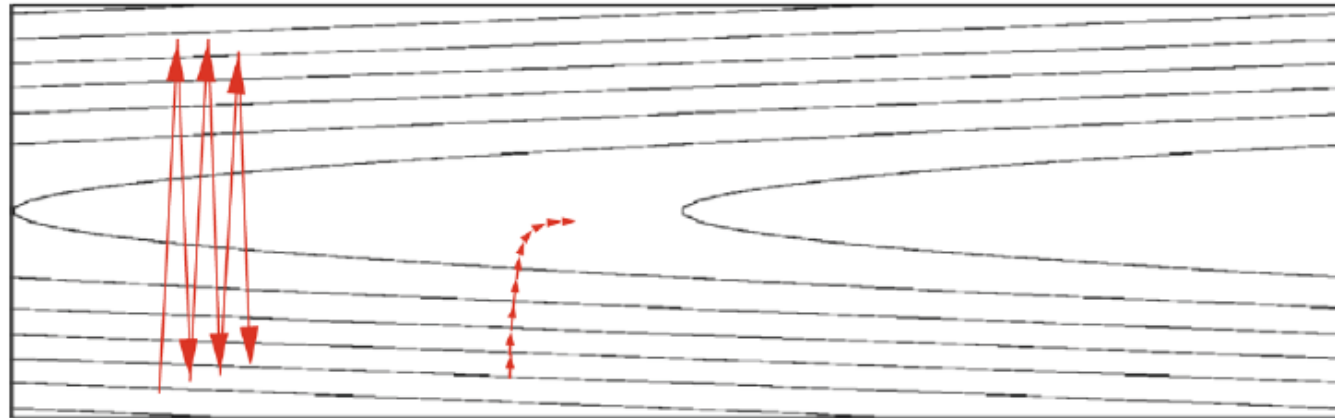


Figure 3.10: Gradient descent for a large η (left) and a very small η (right)

5. Linear Networks with minimal errors – The Delta Rule

- By replacing the activation

$$y^p = \sum_{i=1}^n w_i q_i^p$$

- of the **output neuron** for applied training example q^p , the formula is simplified and we obtain the **delta rule**

$$\Delta w_j = \eta \sum_{p=1}^n (t^p - y^p) q_j^p$$

- Thus for **every training example** the difference between the target t^p and the actual output y^p of the network is calculated for the given input q^p .
- *After summing over all patterns, the weights are then changed proportionally to the sum.*

5. Linear Networks with minimal errors – The Delta Rule

- This algorithm is shown in **Figure 3.11**.
- **Figure 3.11** illustrates the learning of a **two-layer linear network** with the **delta rule**.
- Notice that the **weight changes** always occur **after all of the training data are applied**.

```
DELTALEARNING(TrainingExamples,  $\eta$ )  
Initialize all weights  $w_j$  randomly  
Repeat  
     $\Delta w = 0$   
    For all  $(q^p, t^p) \in \textit{TrainingExamples}$   
        Calculate network output  $y^p = w^p q^p$   
         $\Delta w = \Delta w + \eta(t^p - y^p)q^p$   
     $w = w + \Delta w$   
Until  $w$  converges
```

Figure 3.11: Learning a two-layer linear network with the delta rule.

5. Linear Networks with minimal errors – The Delta Rule

- We see that the algorithm is still **not really incremental** because the weight changes only occur *after all training example have been applied once*.
- We can correct this deficiency by *directly changing the weights* (incremental gradient descent) *after every training example* (Figure 3.12), which, strictly speaking, is no longer a correct implementation of the delta rule.

```
DELTALEARNINGINCREMENTAL(TrainingExamples,  $\eta$ )
Initialize all weights  $w_j$  randomly
Repeat
  For all  $(q^p, t^p) \in \text{TrainingExamples}$ 
    Calculate network output  $y^p = \mathbf{w}^p \mathbf{q}^p$ 
     $\mathbf{w} = \mathbf{w} + \eta(t^p - y^p)\mathbf{q}^p$ 
Until  $\mathbf{w}$  converges
```

Figure 3.12: Incremental variant of the delta rule

AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
2. From Biology to Simulation
3. The Mathematical Model
4. Hopfield Networks
5. Linear Networks with minimal errors
- 6. The Backpropagation Algorithm**
7. Support Vector Machines
8. Deep Learning

AI Fundamentals

Chapter 3: Neural Networks

6. The Backpropagation Algorithm

- With the **backpropagation algorithm**, we now introduce the *most-used neural model*.
- The reason for its widespread use is the *universal versatility* for arbitrary approximation tasks.
- The algorithm originates directly from the **incremental delta rule**.
- In contrast to the delta rule, it applies a **nonlinear sigmoid function** on the weighted sum of the inputs as its **activation function**.
- Furthermore, a backpropagation network can have *more* than two layers of neurons.
- In **Figure 3.13** a typical backpropagation network with an *input layer*, a *hidden layer*, and an *output layer* is shown.

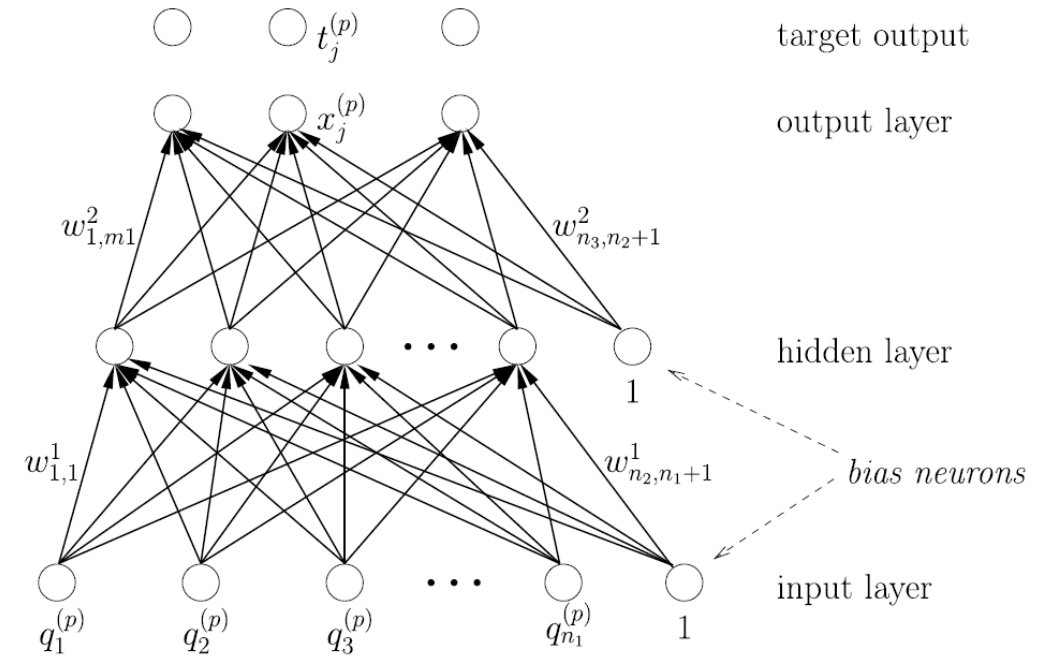


Figure 3.13: A three-layer backpropagation network

6. The Backpropagation Algorithm

- Since the current output value x_j^p of the output layer neuron is compared with the target output t_j^p , these are drawn parallel to each other.
- Other than the input neurons, all neurons calculate their current value x_j by the rule

$$x_j = f\left(\sum_{i=1}^n w_{ji} x_i\right) \quad [3.3]$$

- where n is the number of neurons in the previous layer.
- We use the **sigmoid function**

$$f(x) = \frac{1}{1 + e^{-x}}$$

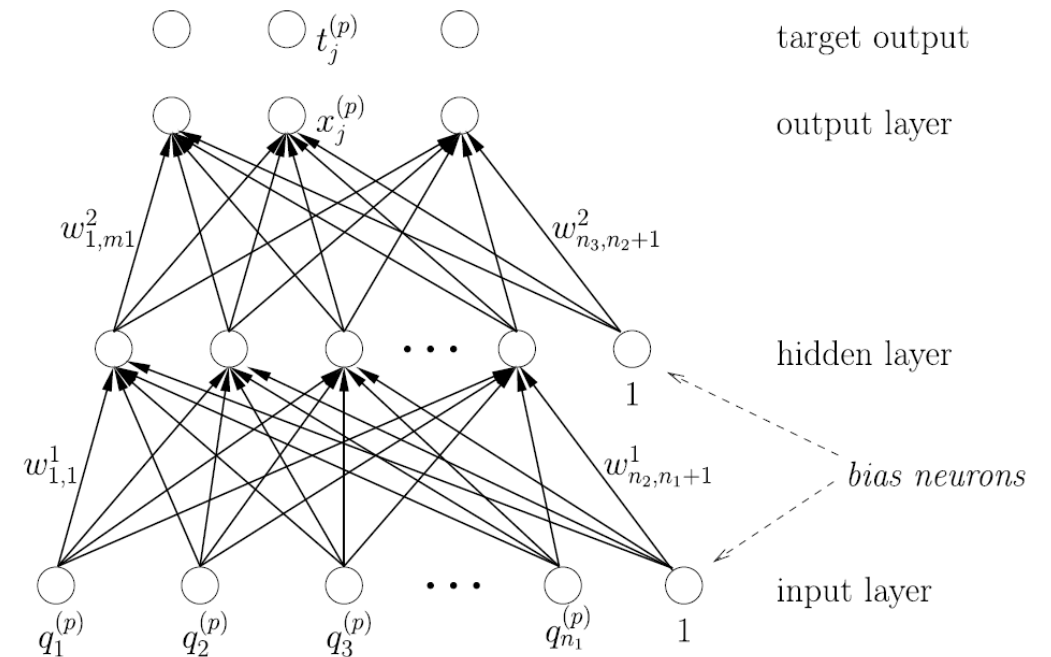


Figure 3.13: A three-layer backpropagation network

6. The Backpropagation Algorithm

- Analogous to the **incremental delta rule**, the weights are changed proportional to the **negative gradient** of the **quadratic error function** summed over the output neurons

$$E_p(w) = \frac{1}{2} \sum_{k \in \text{output}} (t_k^p - x_k^p)^2$$

- For the training pattern p :

$$\Delta_p w_{ji} = -\eta \frac{\partial E_p}{\partial w_{ji}}$$

- For deriving the learning rule, the above expression is substituted for E_p . Within the expression, x_k is replaced by **[3.3]**.
- Within the equation, the outputs x_i of the neurons of the next, deeper layer occur **recursively**.
- By multiple applications of the **chain rule**, we obtain the **backpropagation learning rule**.
- URL: [The Chain rule of differentiation](#)

6. The Backpropagation Algorithm

- By multiple applications of the **chain rule**, we obtain the **backpropagation learning rule**.

$$\Delta_p w_{ji} = \eta \delta_j^{(p)} x_i^{(p)},$$

with

$$\delta_j^{(p)} = \begin{cases} x_j^{(p)}(1 - x_j^{(p)})(t_j^{(p)} - x_j^{(p)}) & \text{if } j \text{ is an output neuron,} \\ x_j^{(p)}(1 - x_j^{(p)}) \sum_k \delta_k^{(p)} w_{kj} & \text{if } j \text{ is a hidden neuron,} \end{cases}$$

- which is also denoted the **generalized delta rule**.

6. The Backpropagation Algorithm

- For all neurons, the formula for changing weight w_{ji} from neuron i to neuron j (see **Figure 3.14**) contains, like the **Hebb rule**, a term $\eta x_i^p x_j^p$
- The new factor $(1 - x_j^p)$ creates the **symmetry**, which is missing from the **Hebb rule**, between the activations 0 and 1 of neuron j .

with

$$\Delta_p w_{ji} = \eta \delta_j^{(p)} x_i^{(p)},$$

$$\delta_j^{(p)} = \begin{cases} x_j^{(p)}(1 - x_j^{(p)})(t_j^{(p)} - x_j^{(p)}) & \text{if } j \text{ is an output neuron,} \\ x_j^{(p)}(1 - x_j^{(p)}) \sum_k \delta_k^{(p)} w_{kj} & \text{if } j \text{ is a hidden neuron,} \end{cases}$$

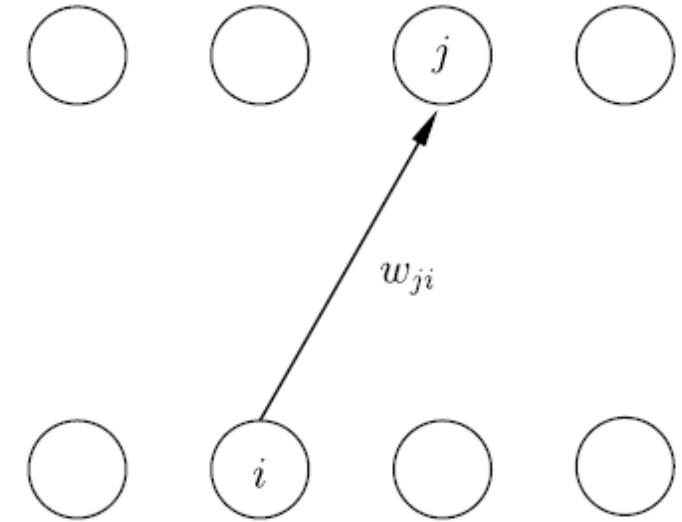


Figure 3.14: Designation of the neurons and weights for the application of the backpropagation rule

6. The Backpropagation Algorithm

- For the output neurons, the factor $(t_j^p - x_j^p)$ takes care of a weight change proportional to the error.
- For the hidden neurons, the value δ_j^p of neuron j is calculated **recursively** from all change δ_k^p of the neurons of the next higher level.

with

$$\Delta_p w_{ji} = \eta \delta_j^{(p)} x_i^{(p)},$$

$$\delta_j^{(p)} = \begin{cases} x_j^{(p)}(1 - x_j^{(p)})(t_j^{(p)} - x_j^{(p)}) & \text{if } j \text{ is an output neuron,} \\ x_j^{(p)}(1 - x_j^{(p)}) \sum_k \delta_k^{(p)} w_{kj} & \text{if } j \text{ is a hidden neuron,} \end{cases}$$

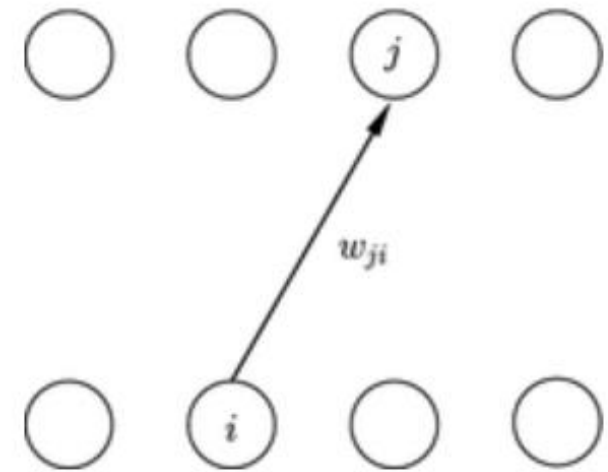


Figure 3.14: Designation of the neurons and weights for the application of the backpropagation rule

6. The Backpropagation Algorithm

- The entire execution of the learning process is shown in **Figure 3.15**.
- After calculation the output of the network (**forward propagation**) for a training example, the approximation error is calculated.
- This is then used during **backward propagation** to alter the weights backward from layer to layer.
- *The whole process is then applied to all training examples and repeated until the weights no longer change or a time limit is reached.*

```
BACKPROPAGATION(TrainingExamples,  $\eta$ )
Initialize all weights  $w_j$  to random values
Repeat
  For all  $(\mathbf{q}^{(p)}, \mathbf{t}^{(p)}) \in \text{TrainingExamples}$ 
    Apply the query vector  $\mathbf{q}^{(p)}$  to the input layer
    Forward propagation:
      For all layers from the first hidden layer upwards
        For all neurons of the layer
          Calculate activation  $x_j = f(\sum_{i=1}^n w_{ji}x_i)$ 
    Calculation of the square error  $E_p(\mathbf{w})$ 
    Backward propagation:
      For all layers from the last downwards
        For all neurons  $i$  of the layer update weights  $w_{ji}$ :
          
$$w_{ji} = w_{ji} + \eta \delta_j^{(p)} x_i^{(p)}$$

  Until  $\mathbf{w}$  converges or time limit is reached
```

Figure 3.15: The backpropagation algorithm

6. The Backpropagation Algorithm

- If we build a network with **at least one hidden layer**, **nonlinear mappings** can be learned.
- *Without **hidden layers**, the output neurons are no more powerful than a **linear neuron**, despite the **sigmoid function**.*
- The reason for this is that the sigmoid function is strictly monotonic.
- The same is true for **multi-layer networks** which only use a **linear function** as an **activation function**, for example the **identity function**.
- This is because chained executions of **linear** mappings is **linear** in aggregate.

```
BACKPROPAGATION(TrainingExamples,  $\eta$ )
Initialize all weights  $w_j$  to random values
Repeat
  For all  $(\mathbf{q}^{(p)}, \mathbf{t}^{(p)}) \in \text{TrainingExamples}$ 
    Apply the query vector  $\mathbf{q}^{(p)}$  to the input layer
    Forward propagation:
      For all layers from the first hidden layer upwards
        For all neurons of the layer
          Calculate activation  $x_j = f(\sum_{i=1}^n w_{ji}x_i)$ 
    Calculation of the square error  $E_p(\mathbf{w})$ 
    Backward propagation:
      For all layers from the last downwards
        For all neurons  $i$  of the layer update weights  $w_{ji}$ :
          
$$w_{ji} = w_{ji} + \eta \delta_j^{(p)} x_i^{(p)}$$

  Until  $\mathbf{w}$  converges or time limit is reached
```

Figure 3.15: The backpropagation algorithm

6. The Backpropagation Algorithm

- Just like with the **perceptron**, the class of the functions which can be represented are also enlarged if we use a **variable sigmoid function**

$$f(x) = \frac{1}{1 + e^{-(x-\theta)}}$$

- with **threshold** θ .
- This is implemented analogously to the way shown in **chapter 2** (The Perceptron, a Linear Classifier), in which a neuron whose activation always has the value **one**, and which is connected to neurons in the next highest level is inserted into the input layer and into each hidden layer.
- The weights of these connections are learned normally and represent the threshold θ of the successor neurons.

BACKPROPAGATION(*TrainingExamples*, η)

Initialize all weights w_j to random values

Repeat

For all $(\mathbf{q}^{(p)}, \mathbf{t}^{(p)}) \in \text{TrainingExamples}$

Apply the query vector $\mathbf{q}^{(p)}$ to the input layer

Forward propagation:

For all layers from the first hidden layer upwards

For all neurons of the layer

 Calculate activation $x_j = f(\sum_{i=1}^n w_{ji}x_i)$

Calculation of the square error $E_p(\mathbf{w})$

Backward propagation:

For all layers from the last downwards

For all neurons i of the layer update weights w_{ji} :

$$w_{ji} = w_{ji} + \eta \delta_j^{(p)} x_i^{(p)}$$

Until $\mathbf{w}$ converges or time limit is reached

Figure 3.15: The backpropagation algorithm

AI Fundamentals

Chapter 3: Neural Networks

Contents

1. Introduction
2. From Biology to Simulation
3. The Mathematical Model
4. Hopfield Networks
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
- 7. Support Vector Machines**
8. Deep Learning

AI Fundamentals

Chapter 3: Neural Networks

7. Support Vector Machines

- Feedforward neural networks with **only one layer** of weights are **linear**.
- **Linearity** leads to **simple networks** and **fast learning** with **guaranteed convergence**.
- Furthermore, the danger of **overfitting** is *small* for **linear** models.
- For many applications, however, the linear models are not strong enough, for example because the relevant classes are *not linearly separable*.
- Here **multi-layered networks** such as **backpropagation** come into use, with the consequence that **local minima**, **convergence problems** and **overfitting** can occur.

7. Support Vector Machines

- A promising approach, which brings together the advantages of **linear** and **nonlinear** models, follows the theory of **support vector machines** (SVM), which we will roughly outline using a two class problem.
- In the case of two *linearly* separable classes, it is easy to find a dividing hyper plane, for example with the **perceptron learning rule**.
- However, there are usually *infinitely many such planes*, as in the two-dimensional example in **Figure 3.16**. **Figure 3.16** illustrates *two classes* with the maximally dividing line. The circled points are the *support vectors*.

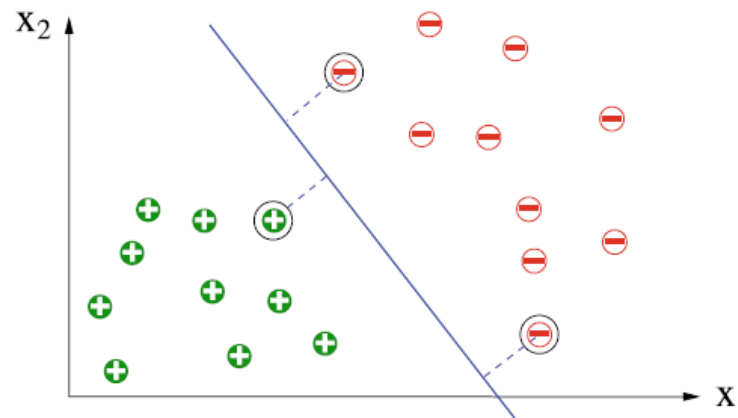


Figure 3.16: Two classes with the maximally dividing line

7. Support Vector Machines

- **Support vector machines** apply this algorithm to **non linearly separable problems** in a *two-step process*.
- In the **first step**, a **nonlinear** transformation is applied to the data, with the property that the transformed data is **linearly separable**.
- In the **second step** the **support vectors** are then determined in the transformed space.
- The first step is highly interesting, but not quite simple.
- *In fact, it is always possible to make the classes **linearly** separable by transforming the vector space, as long as the data contains no contradictions.*
- Such a separation can be reached for example by introducing a new $(n + 1)th$ dimension and the definition

$$x_{n+1} = \begin{cases} 1 & \text{if } x \in \text{class 1,} \\ 0 & \text{if } x \in \text{class 0.} \end{cases}$$

URL: [Support vector machine](#)

7. Support Vector Machines

- Such a separation can be reached for example by introducing a new $(n + 1)th$ dimension and the definition

$$x_{n+1} = \begin{cases} 1 & \text{if } x \in \text{class 1,} \\ 0 & \text{if } x \in \text{class 0.} \end{cases}$$

- However, this formula does not help much because it is not applicable to new points of an unknown class which are to be classified.
- *We thus need a transformation which is as **independent** as possible from the current data.*
- *It can be shown that there are such generic transformations even for arbitrarily shaped class division boundaries in the original vector space.*
- In the transformed space, the data are then **linearly** separable.
- However, the number of dimensions is not so problematic because, when using **support vectors**, the dividing plane, as mentioned above, is determined by only a few parameters.
- The central **nonlinear transformation** of the vector space is called the **kernel**, because of which support vector machines are also known as **kernel methods**.
- The original SVM theory developed for **classification** tasks has been extended and can now be used on **regression** problems also.

AI Fundamentals

Chapter 3: Neural Networks

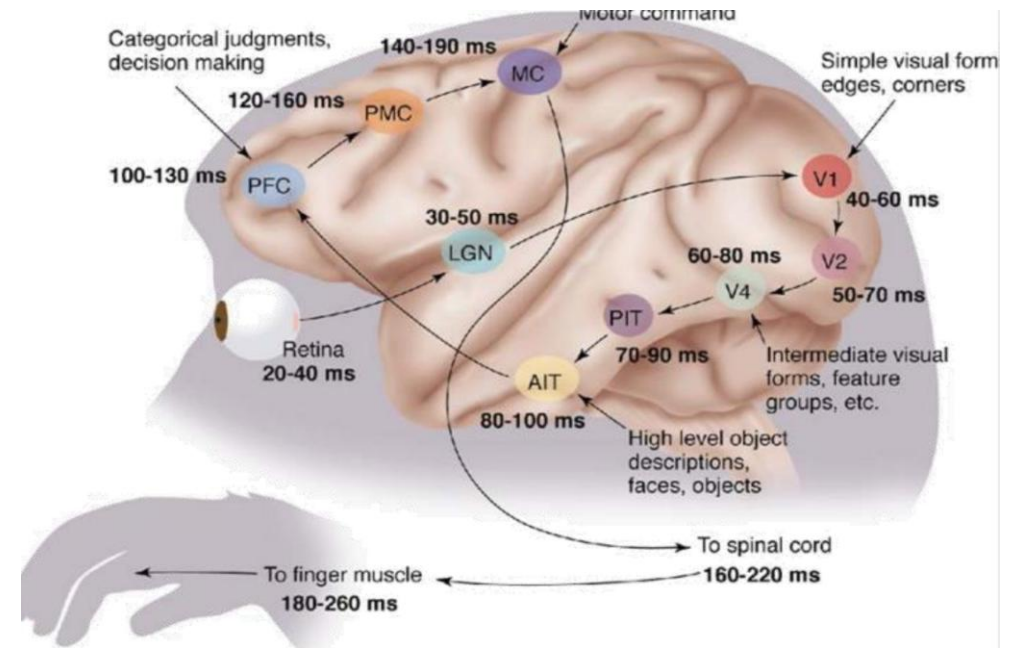
Contents

1. Introduction
2. From Biology to Simulation
3. The Mathematical Model
4. Hopfield Networks
5. Linear Networks with minimal errors
6. The Backpropagation Algorithm
7. Support Vector Machines
- 8. Deep Learning**

AI Fundamentals

Chapter 3: Neural Networks

Deep Learning: inspired by the brain

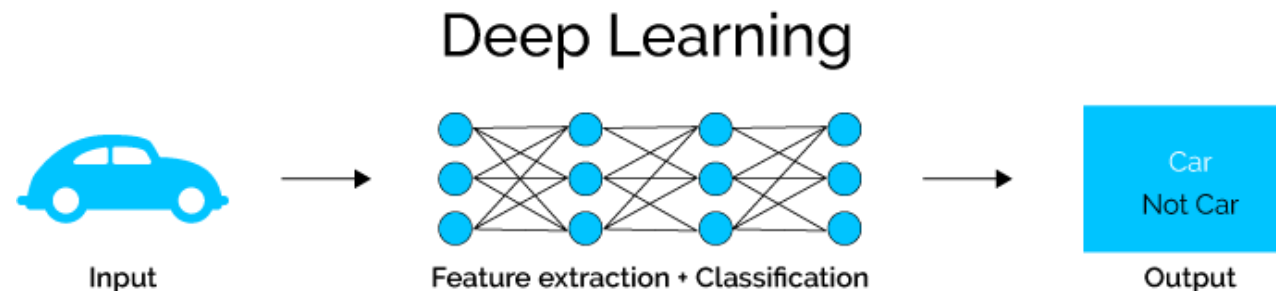
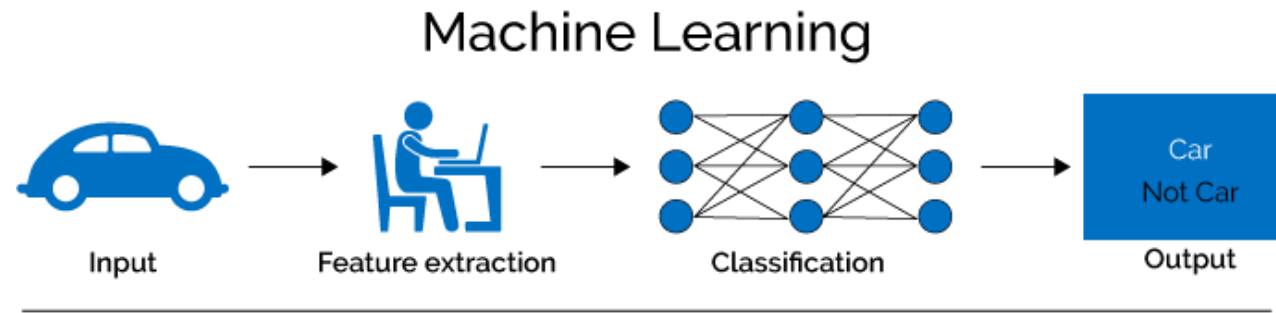


URL: [Inspired by the brain](#)

- The first hierarchy of neurons that receives information in the visual cortex are sensitive to specific edges while brain regions further down the visual pipeline are sensitive to more complex structures such as faces.
- Our brain has lots of neurons connected together and the strength of the connections between neurons represents long term knowledge.
- One learning algorithm hypothesis: all significant mental algorithms are learned except for the learning and reward machinery itself.

What is deep learning?

- Subfield of Machine Learning for **learning representations** of data.
- Exceptional effective at **learning patterns**.
- Utilizes learning algorithms that derive **meaning** out of data by using a **hierarchy of multiple layers** that mimic the **neural networks of our brain**.
- If you provide the system **lots of information**, it begins to understand it and respond in useful ways.



URL: [Deep learning vs. machine learning](#)

AI Fundamentals

Chapter 3: Neural Networks

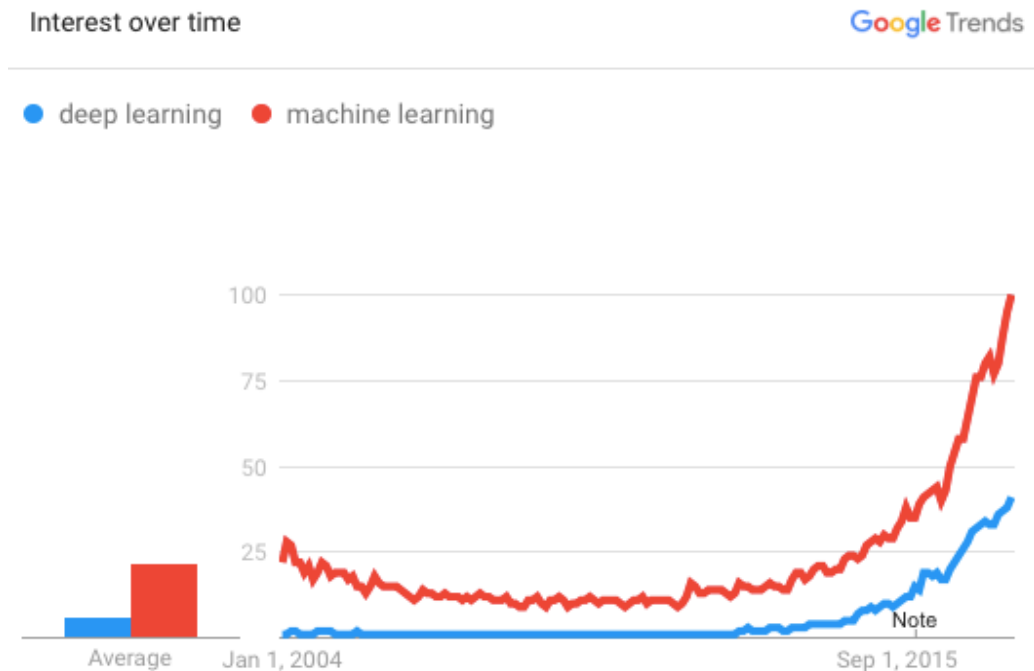
Why is Deep Learning useful?

- Manually designed features are often **over-specified**, **incomplete** and take a **long time to design** and validate
- Learned Features are **easy to adapt**, **fast** to learn
- Deep learning provides a very **flexible**, (almost?) **universal**, learnable framework for representing world, visual and linguistic information.
- Can learn both unsupervised and supervised
- Effective **end-to-end** joint system learning
- Utilize large amounts of training data

In ~2010 Deep Learning started outperforming other Machine Learning techniques first in speech and vision

URL: [Deep learning vs. machine learning](#)

URL: [Deep learning vs. machine learning - Google Trends](#)



Deep learning: applications

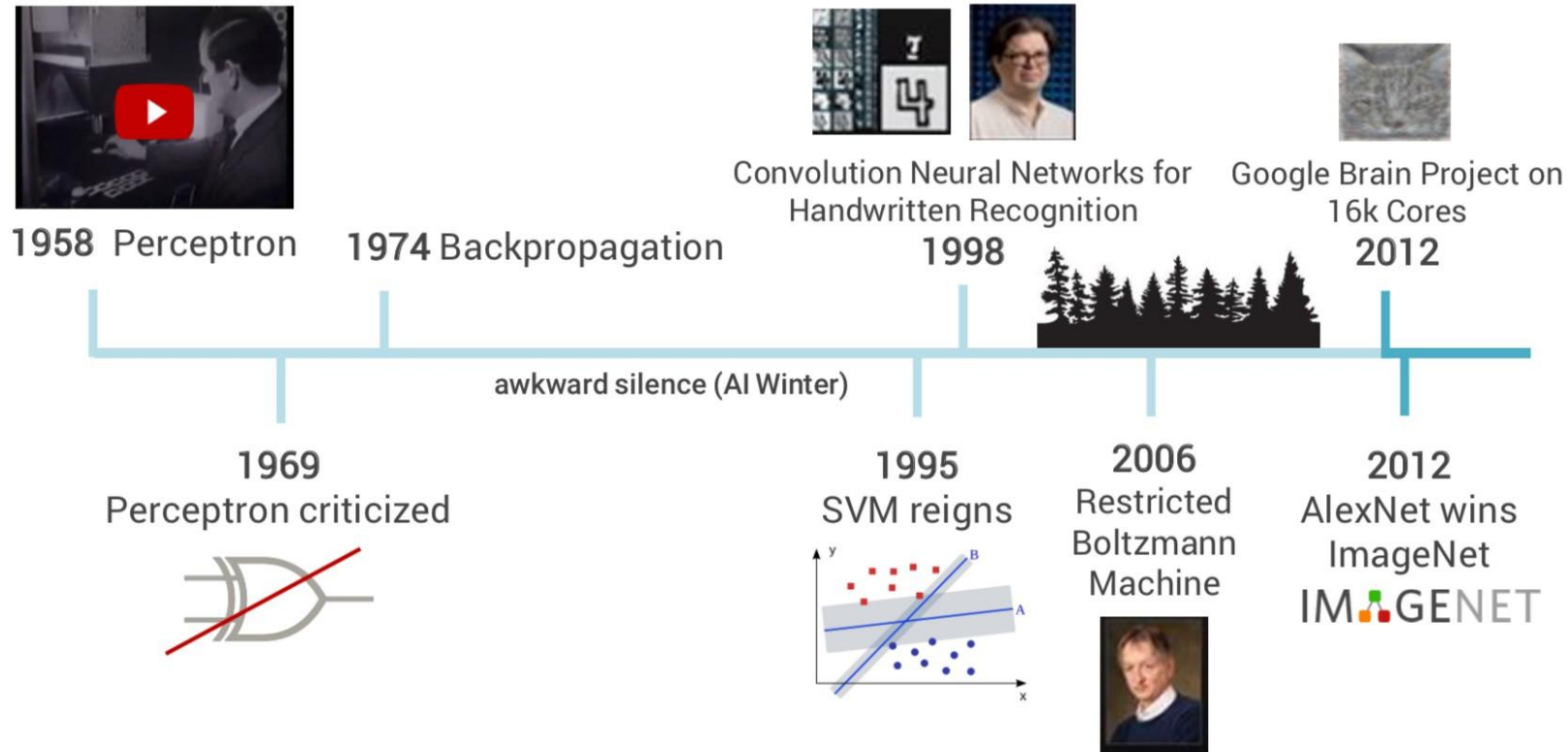


- Face recognition (Facebook, tagging!)
- Speech recognition
- Natural Language Processing (NLP)
- ...
- Even in your mobile device
 - Siri
 - Google assistant

AI Fundamentals

Chapter 3: Neural Networks

Deep learning: brief history



AI Fundamentals

Chapter 3: Neural Networks

Deep Learning : brief history



ImageNet: The “computer vision World Cup”

URL: [A brief history](#)

8. Deep Learning

- We saw that there are many **good learning algorithms** today which are capable of learning non-trivial, sometimes complex, classifications or approximations for all sort of applications, such as diagnosis and prognosis based on sensor inputs.
- We have also seen that, up to now, generation of features has not been successful. Instead it is the job of a **data scientist** to find a sensible small set of features, which can then be used as input for the learning algorithm.
- *Why do we not simply use all available sensor data, i.e. a direct image of the world, as input?*
- For example, for object recognition in a photo we could use all ten million pixels as an input vector, which has a length of 30 million (in the case of RGB or HSV images, which each have three color values).
- *What is the problem with this approach?*
- The answer is known as the “*curse of dimensionality*”.

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning

- *What is the problem with this approach?*
- The answer is known as the “*curse of dimensionality*”.
- This means, among other things, that **training time** grows very fast, often *exponentially*, with the dimension of the input data.
- To keep computation time within bounds, we must therefore first reduce the input data to short feature vectors.
- *As described above, this mapping is usually created **manually**.*
- However, for many applications, such as object classification in images it is difficult if not **impossible to manually find the formula for features**.
- One old method for automatic reduction of dimensions is **principal component analysis** (PCA), which determines the directions of the **highest variance** (i.e. the principal component) in the vector space of the training data and projects the data into the subspace of the principal components using a **linear transformation**.
- Because of the missing nonlinearity of the compressing mapping, PCA is not as powerful as the new methods described below.

URL: [Hoofdcomponentenanalyse](#)

8. Deep Learning

- Since about **1995** work has been done on **deep learning**, a highly promising class of algorithms to solve this problem, and there are now impressive successes to report.
- **Deep learning** includes methods such as **convolutional neural networks** (CNN).
- The architectures of the **multi-layered neural networks** with up to twenty or more layers are in part very complex and cannot be explained in detail here.
- **Pattern recognition** is simple in low-dimensional spaces or in case of **classification** when the classes are **linearly separable**.
- For classes that are **not linearly separable** in high dimensional spaces, however, problems arise because here learning poses a **nonlinear optimization problem**.
- In principle there are solutions using **gradient descent algorithms** such as **backpropagation**.
- However, convergence problems and unacceptably high computation times arise for the classical algorithms, especially when networks with many hidden layers are used.

URL: [Convolutional neural network](#)

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Nature as Example

- All successful approaches in deep learning to date work with many layers of **neurons**.
- The network is split into two parts, similar to the **feedforward network** shown in **Figure 3.17**. It is composed of a *preprocessor*, *several layers* (in this case two) for **unsupervised** detection and then one classical neural network, which, in this example, is trained by **backpropagation**.
- After a *preprocessing* layer there are several layers which are pre-trained by **unsupervised learning** (UL).
- Each layer in this UL network represents features of the input pattern.
- The lower the layer, the simpler the features.
- In **object recognition** in photos, the features of the lower layers typically represent *edges* or *lines* in different orientations. *Complex features* such as the presence of a face can form on **higher layers**.

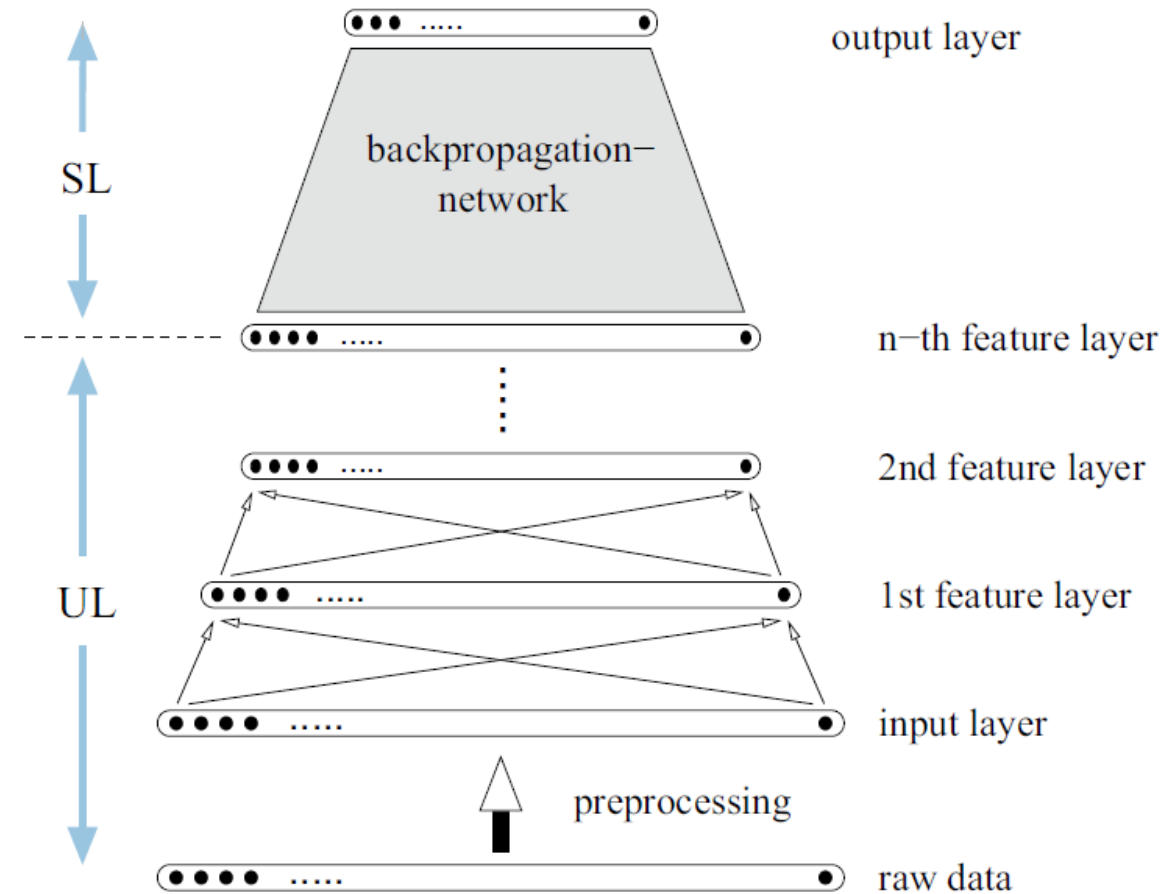


Figure 3.17: Simplified architecture of a deep learning network

8. Deep Learning – Nature as Example

- This architecture shows certain similarities to the structure of the **brains** of humans and animals.
- Starting from the sense organs, for example the eyes; the brain is built up in many layers, and the higher the layer, the more abstract the information to be found there.
- However, there is still far too little known about how neural networks work in nature that leads to a significant benefit in deep learning.

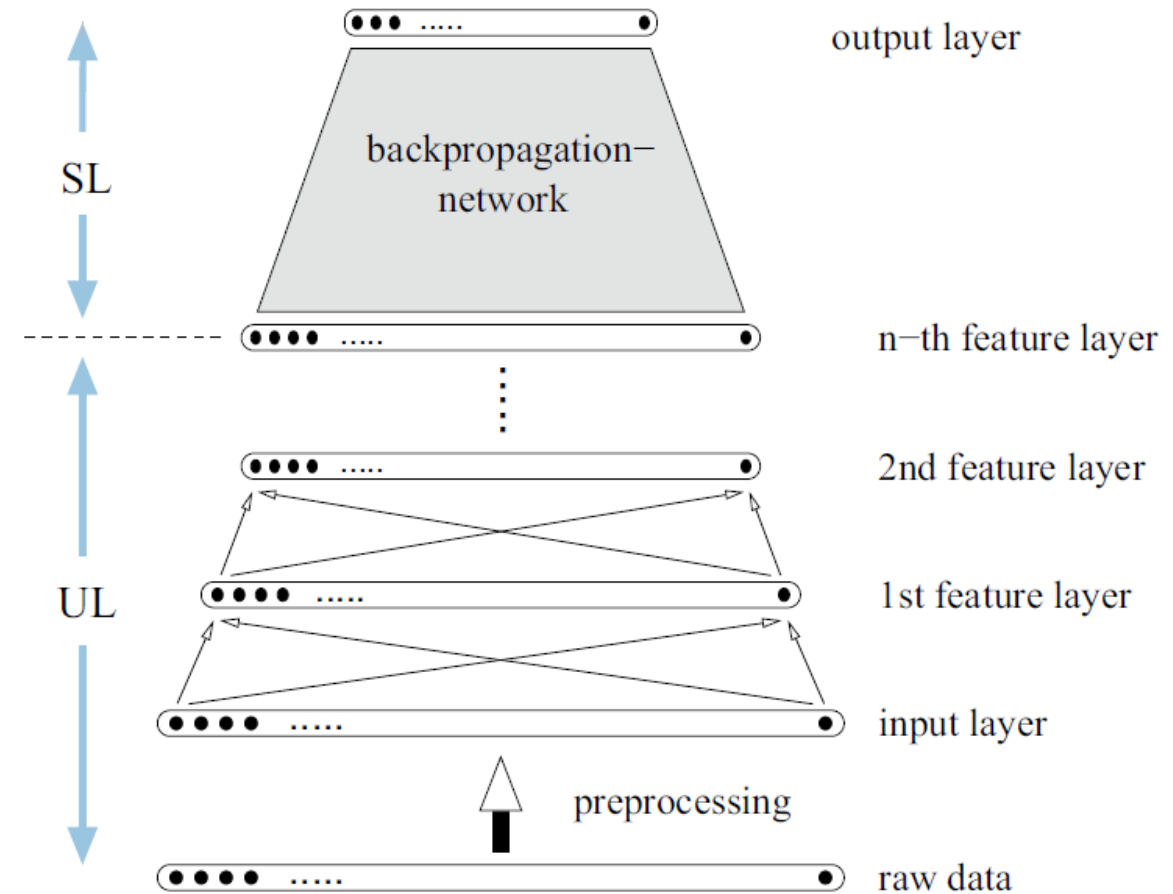


Figure 3.17: Simplified architecture of a deep learning network

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Nature as Example

- Attached to the UL network is a classical **supervised learning** (SL) network, which can be trained with backpropagation.
- Thus the learning process works as follows:
 1. **Unsupervised** training of all feature layer weights
 2. **Supervised** learning of the SL network with gradient descent.
- The unsupervised learning of the weights to the feature layers remains to be explained.
- By learning these weights, the features will be extracted.
- Feature extraction should have the property that input data is mapped into a lower dimensional space, if possible without (too much) loss of information.
- One could see **feature extraction** as a form of **compression**.

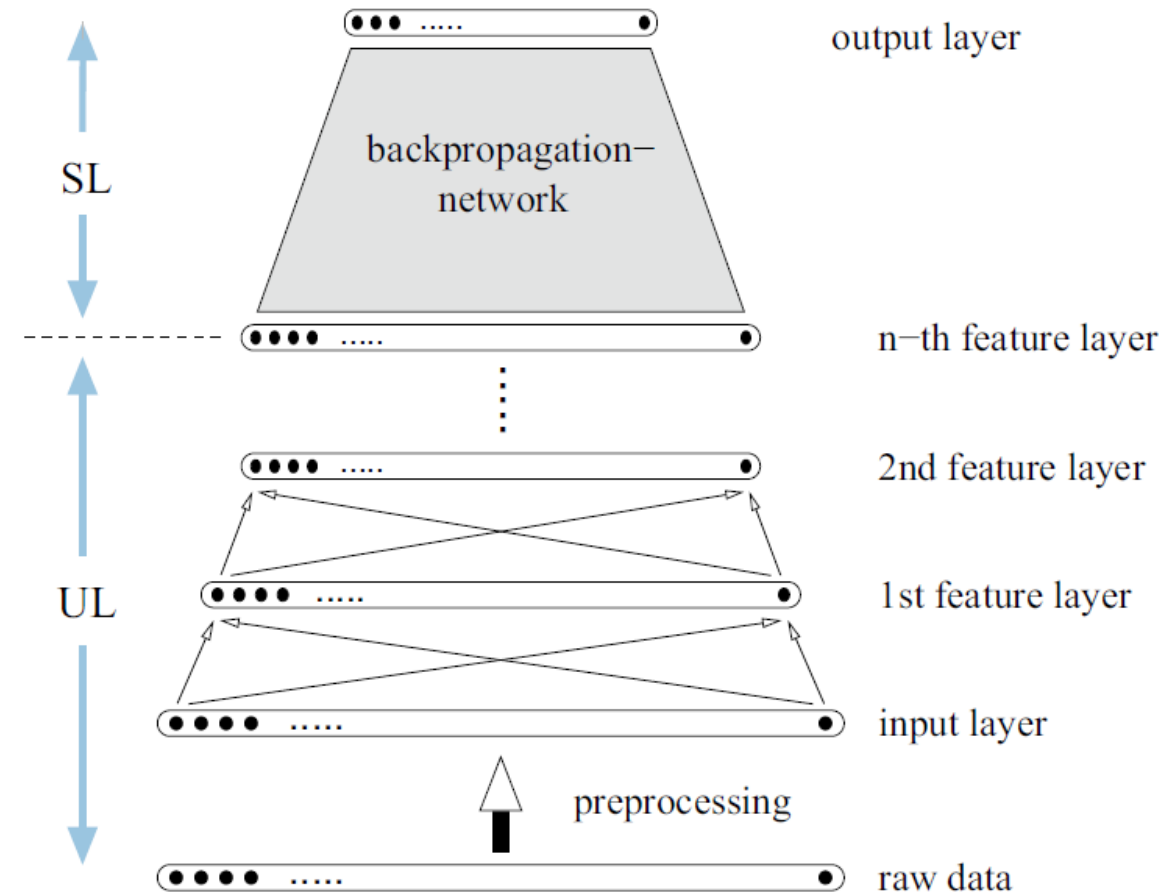


Figure 3.17: Simplified architecture of a deep learning network

8. Deep Learning – Stacked Denoising Autoencoder

- The feature layer's weights are determined by a fundamentally simple and obvious algorithm called **stacked denoising autoencoder**, as shown in **Figure 3.18**.
- **Figure 3.18** illustrates an autoencoder for learning the weights of a feature layer. A classical **backpropagation** network can be used here to learn the **identity mapping** from the current input layer onto itself. **Features** form in the hidden layer during learning.

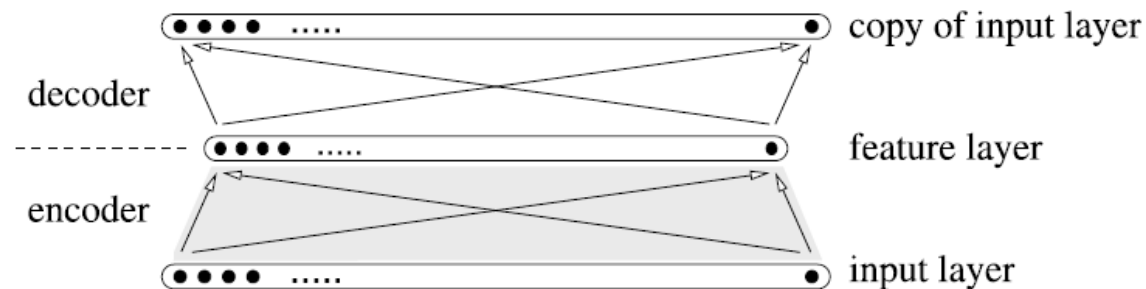


Figure 3.18: Autoencoder for learning the weights of a feature layer

- URL: [Autoencoder](#)
- URL: [Denoising Autoencoder \(DAE\)](#)

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Stacked Denoising Autoencoder

- To train the first hidden layer, an autoencoder is trained with a **supervised** learning method.
- *Its job is to learn the identity mapping of all input vectors x onto itself.*
- The feature layer acts as the hidden layer.
- To make sure that features form in this layer, it is important **to avoid overfitting**.
- The classical approach would be to use **cross-validation** over the number of hidden neurons in the feature layer.
- That would lead to a small number of hidden neurons and thus new features.
- URL: [Cross Validation in Machine Learning](#)

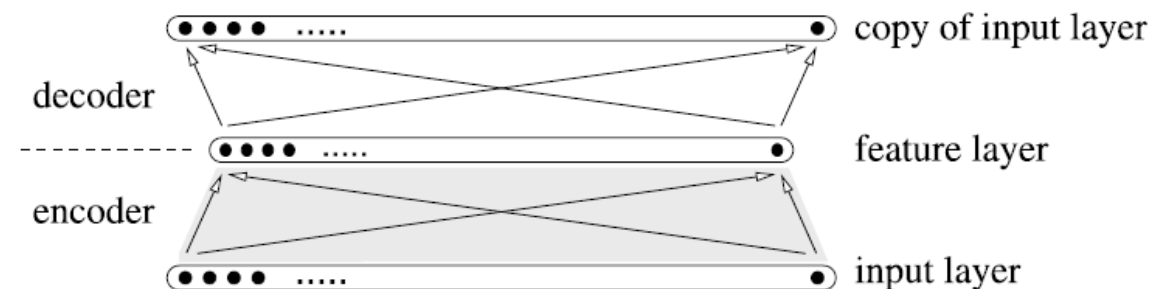


Figure 3.18: Autoencoder for learning the weights of a feature layer

8. Deep Learning – Stacked Denoising Autoencoder

- In order to find many different features **denoising** rather than **cross-validation** will be used.
- During each learning cycle, the values of some of the input neurons are randomly changed, for example with **Gaussian noise** or by setting the **activation** to 0 or 1.
- The **distance** $\|y - x\|$ of the calculated output vector y from the original input vector x is used as the **error function** for the weight change.
- Thus the **encoder** is trained to become **robust against noise**.
- During learning, a variable parameter weights the noisified neurons more strongly than the others, so as to optimize the desired effect of the noise on learning.

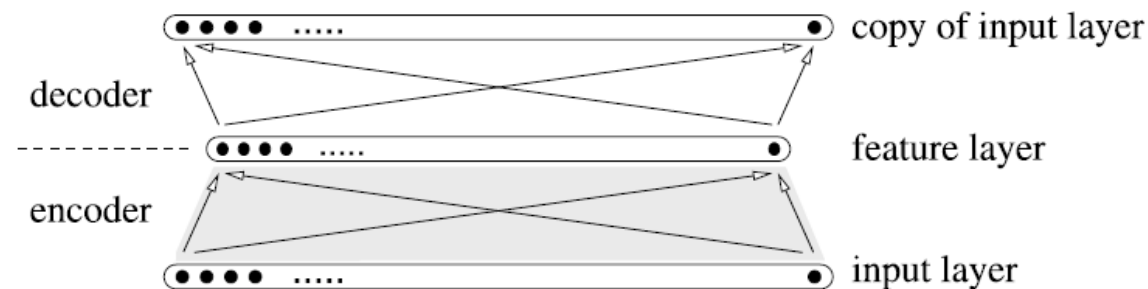


Figure 3.18: Autoencoder for learning the weights of a feature layer

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning

Stacked Denoising Autoencoder

- After the autoencoder is trained, the actually unneeded decoder layer, i.e. the second layer of weights, is *removed*.
- *The first layer is frozen and used to calculate the features in the UL network.*
- Then, with this fixed first layer of weights, the **second** feature layer is trained with the autoencoder algorithm, frozen, and so on until the last feature layer.
- Thus the **unsupervised** part of the learning is finished.

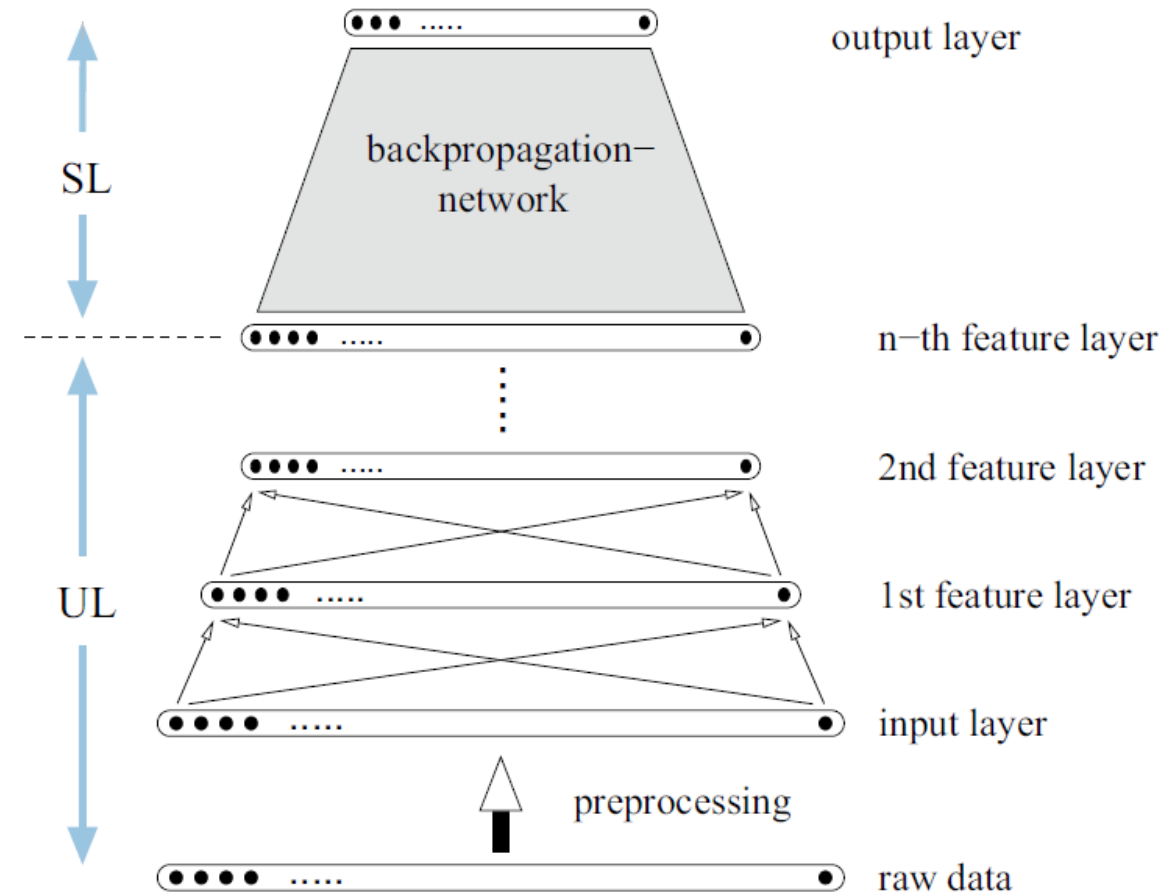


Figure 3.17: Simplified architecture of a deep learning network

8. Deep Learning

Stacked Denoising Autoencoder

- Now the **SL** portion of the network is trained with a **classical supervised learning algorithm**.
- For each training example, the output of the **UL** network, i.e. the last feature layer is used as input, and a particular data label is the target output.
- During **backpropagation**, the weights of the feature layers can be trained again for the purpose of fine-tuning, or they can be left unchanged.

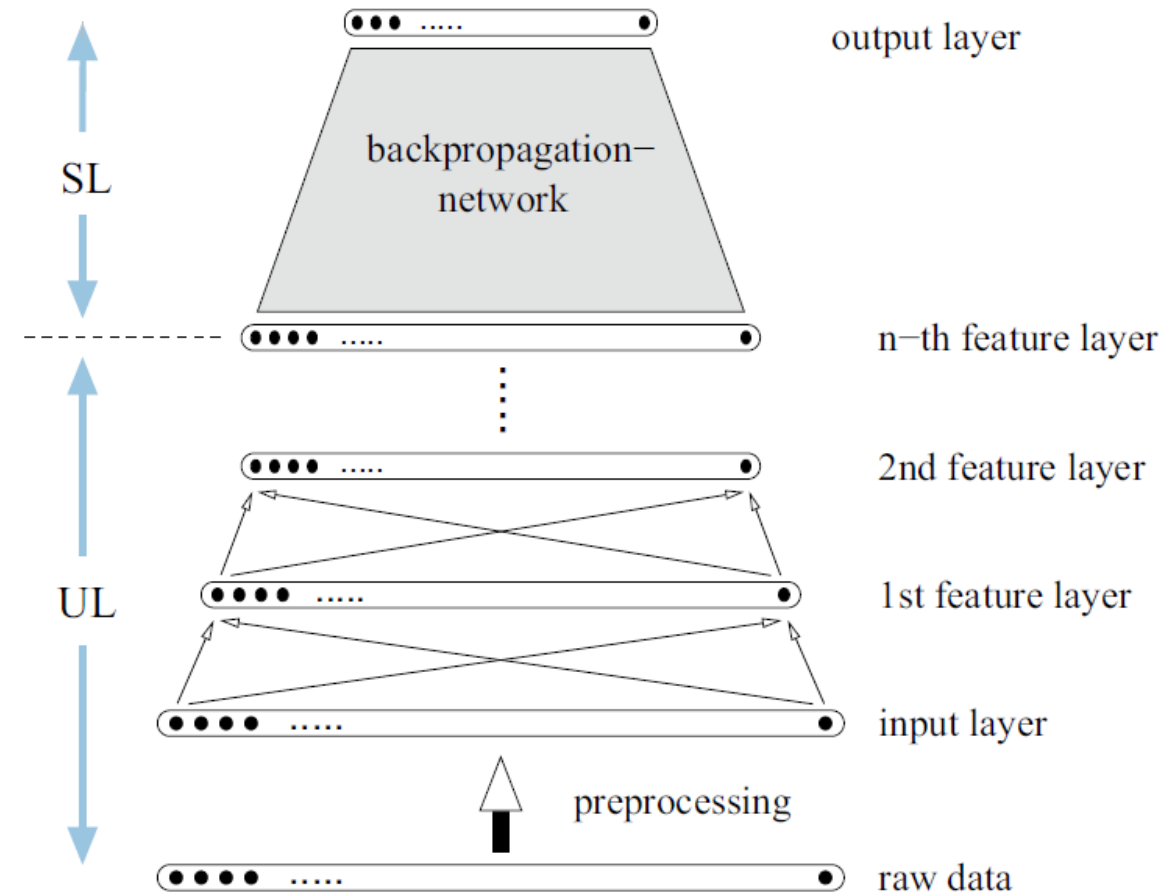


Figure 3.17: Simplified architecture of a deep learning network

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning

Stacked Denoising Autoencoder

- We have not yet mentioned the **preprocessing** step, which differs depending on the application.
- Often all of the input variables are **normalized**.
- For **photos**, the pixel image is often transformed into a lower dimensional space, similar to PCA, using a process called *whitening*, with the goal of making the generated features less redundant.
- Here one could also directly use PCA.
- URL: [PCA Whitening](#)

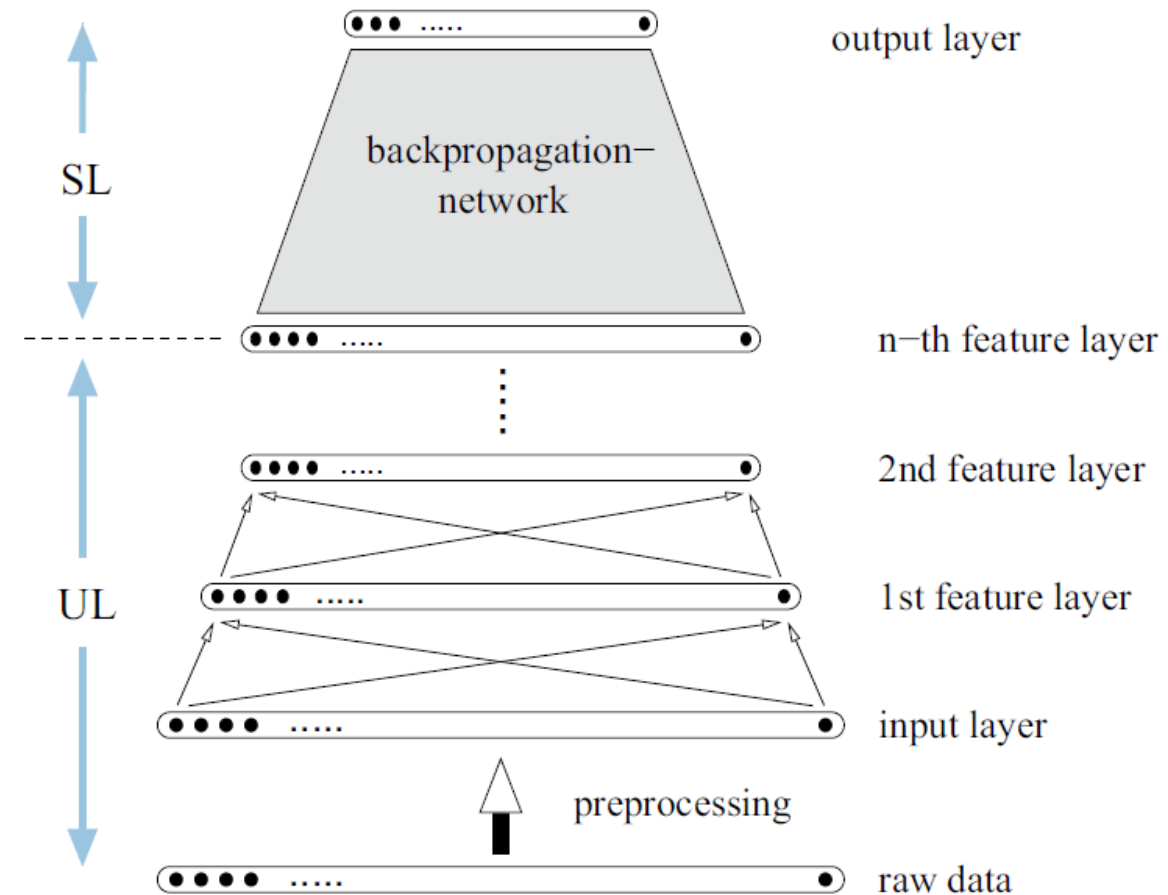


Figure 3.17: Simplified architecture of a deep learning network

8. Deep Learning – Systems and Implementation

- Even the best current implementations of **deep learning** systems are very **computationally intensive**.
- The cause of the **long computation times** is the **size of the input layer** and **the high number of layers** in the network.
- This effect is amplified further if, in the case of a large input layer, the training data are elements of a high-dimensional vector space.
- To represent the trained classes well, a great many data vectors are needed, which makes the computation time even longer.
- *This means that a training run can take from minutes up to even days.*
- In addition, the system parameters of the complex networks have to be configured, which, in turn, has a big impact on the quality of the results.
- **Optimal meta-parameters** for a learning algorithm can be found with **cross-validation**, that is by trying out all combinations of values.
- URL: [Cross Validation in Machine Learning](#)

8. Deep Learning – Systems and Implementation

- Due to the complexity of deep learning algorithms, there are **many parameters**, and the set of parameter combinations grows exponentially with the number of parameters.
- *Therefore, the naïve application of cross-validation is not practical.*
- Instead, algorithms are used which look for a point in the space of the system's **meta-parameters** that **minimizes errors on the validation data**.
- Examples of such **meta-parameters** are the **number of layers** in the UL network as well as the SL network, the **number of neurons** in the individual layers, the **learning rates**, and the **degree of interconnections** for CNN networks.
- The algorithms for **optimization** of meta-parameters use, for example, **random search** or **gradient descent** in the space of meta-parameters.
- Thus, for high-dimensional data sets, simple PCs are overwhelmed.
- Today one works with multiprocessor machines and the learning algorithms are run highly **parallel** on modern **graphic cards**, which ultimately leads to training times of hours to weeks for training with hyper parameter optimization.

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Applications of Deep Learning

- Especially in object classification in photos, deep learning has shown impressive progress.
- For example, in the recognition of hand-written digits, the classification accuracy of **99,8%** has been achieved on the **MNIST** data set (**Figure 3.19** (left)).
- On the **SVHN** data set (**Figure 3.19** (center)), generated as part of the Google Street View Project with photos of home address number, the accuracy is about **98,3%**.
- It is even possible to describe photos in complete sentences. For the foto shown in the right of *Figure 3.19*, the system produces the sentence “*A person riding a motorcycle on a dirt road*”. It uses a **convolutional neural network** for **object recognition** and a **recurrent neural network** for **text generation**.



Figure 3.19: Examples of MNIST data (left, SVHN data (center) and an example photo for the image description system

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Applications of Deep Learning

- In addition to the applications given as examples thus far, there are **countless applications** of neural networks in all areas of industry, especially for deep learning.
- **Pattern recognition** in all of its forms is a very important area, whether analysis of photos to recognize people or **faces**, recognition of fish swarms in **sonar readings**, recognition and classification of military vehicles in **radar scans**, or any number of other applications.
- Neural networks can also be trained to **recognize spoken language** and **hand written text**.
- Neural networks are not only used for **recognizing objects** and **scenes**.
- They can be trained **to control self driving cars** or **robots** based on sensor data, as well as for **heuristically controlling search** in backgammon and chess computers.

AI Fundamentals

Chapter 3: Neural Networks

8. Deep Learning – Applications of Deep Learning

- For quite some time, neural networks, in addition to statistical methods, have been used successfully to **forecast stock prices** and to judge the **creditworthiness of bank customers**.
- **Speed trading** of international financial transactions would be impossible without the help of smart and fast neural networks that autonomously decide about buying or selling.
- Other machine learning algorithms can as well be used for many of these applications.
- Due to the great commercial success of **data mining**, **decision tree learning** and **support vector machines**, there are neural algorithms for many applications as well as others that are not biologically motivated at all.
- The field of **neural networks** is a **subarea** of **machine learning**.

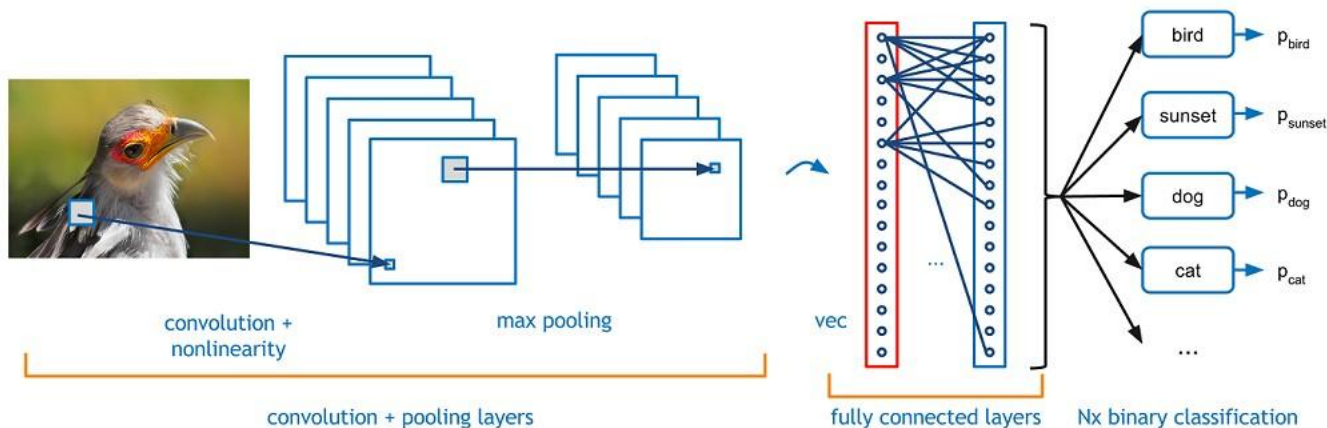
AI Fundamentals

Chapter 3: Neural Networks

Deep Learning: Network Types – CNN vs. RNN

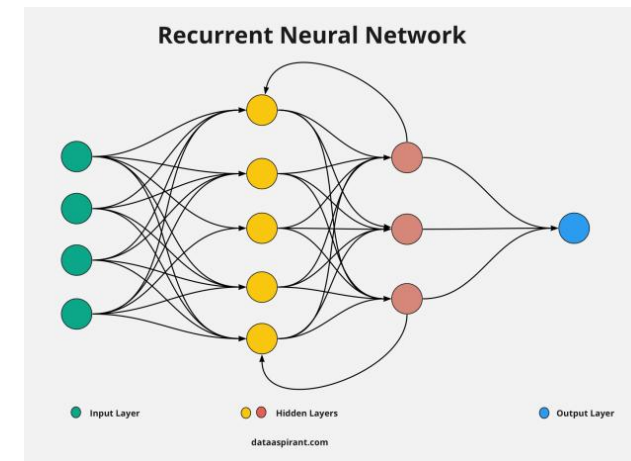
Convolutional Neural Networks (CNN)

- Mainly “visual” problems
- Used by
 - Facebook for tagging
 - Google for photo search



Recurrent neural networks (RNN)

- Sometimes you need memory:
 - Processing sequences
 - Temporal series of data points
 - E.g. reading: you process word-by-word while keeping track of what you have read

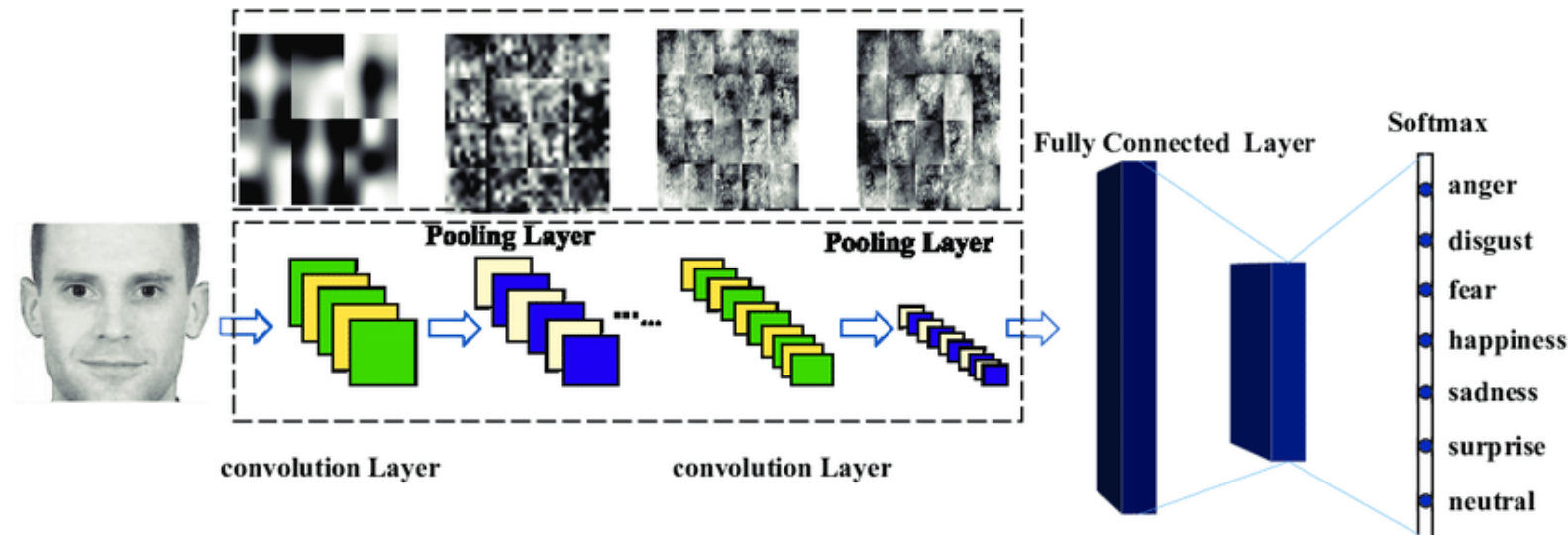


URL: [A beginner's guide to understanding CNN](#)

URL: [Recurrent Neural Network \(RNN\)](#)

Deep Learning: Network Types - Convolutional Neural Networks (CNN)

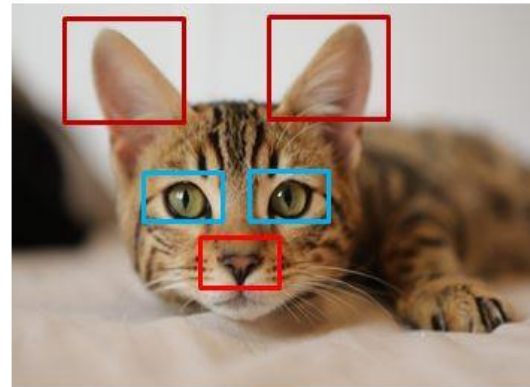
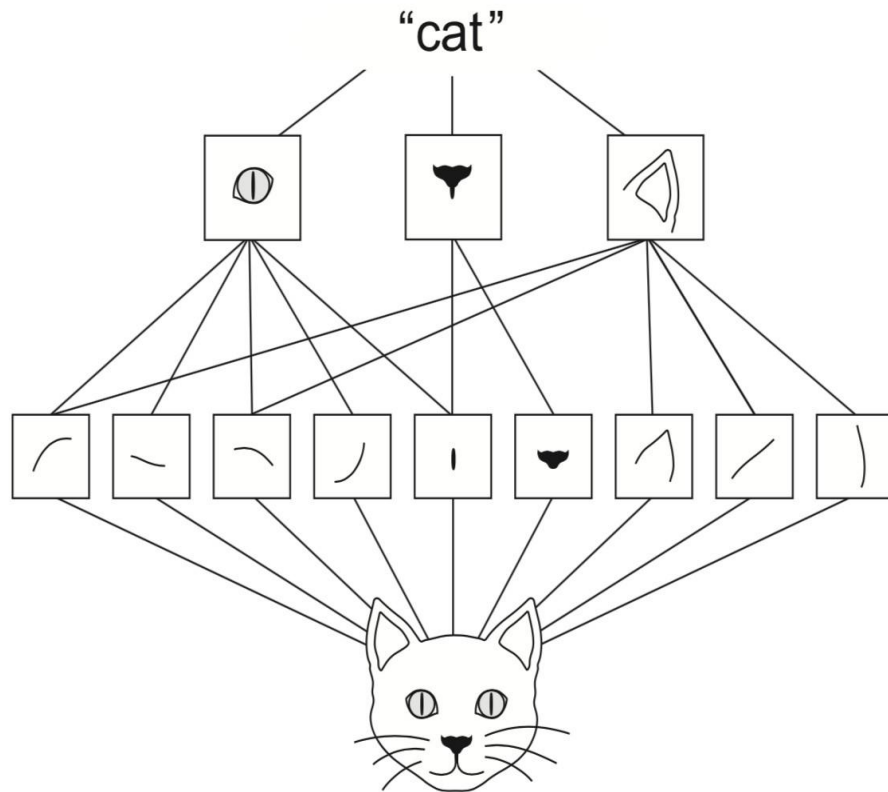
- **Convolutional Neural Networks** (CNN) learn a complex representation of **visual data** using vast amounts of data. They are inspired by the **human visual system** and learn multiple layers of transformations, which are applied on top of each other to extract a progressively more sophisticated representation of the input.



AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): 2D feature detection



Nose Filter
Eyes Filter
Ears Filter

URL: [Neural networks for spatial data](#)

URL: [Image Classification using CNN](#)

AI Fundamentals

Chapter 3: Neural Networks

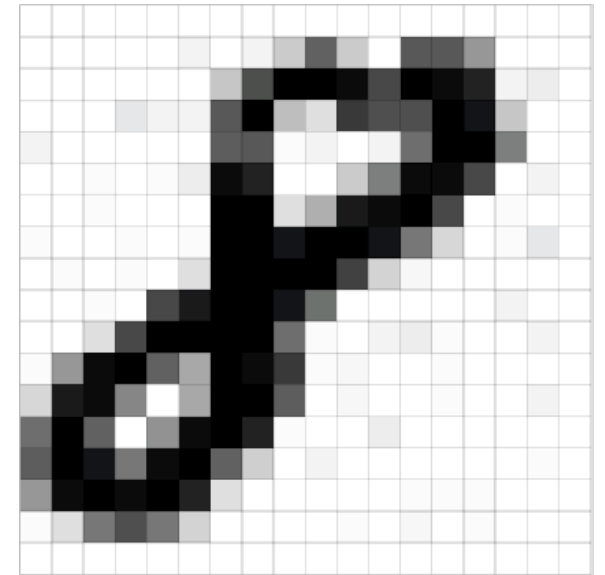
Convolutional Neural Network (CNN): 2D image representation



What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 90
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 26 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
66 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48
```

What Computers See

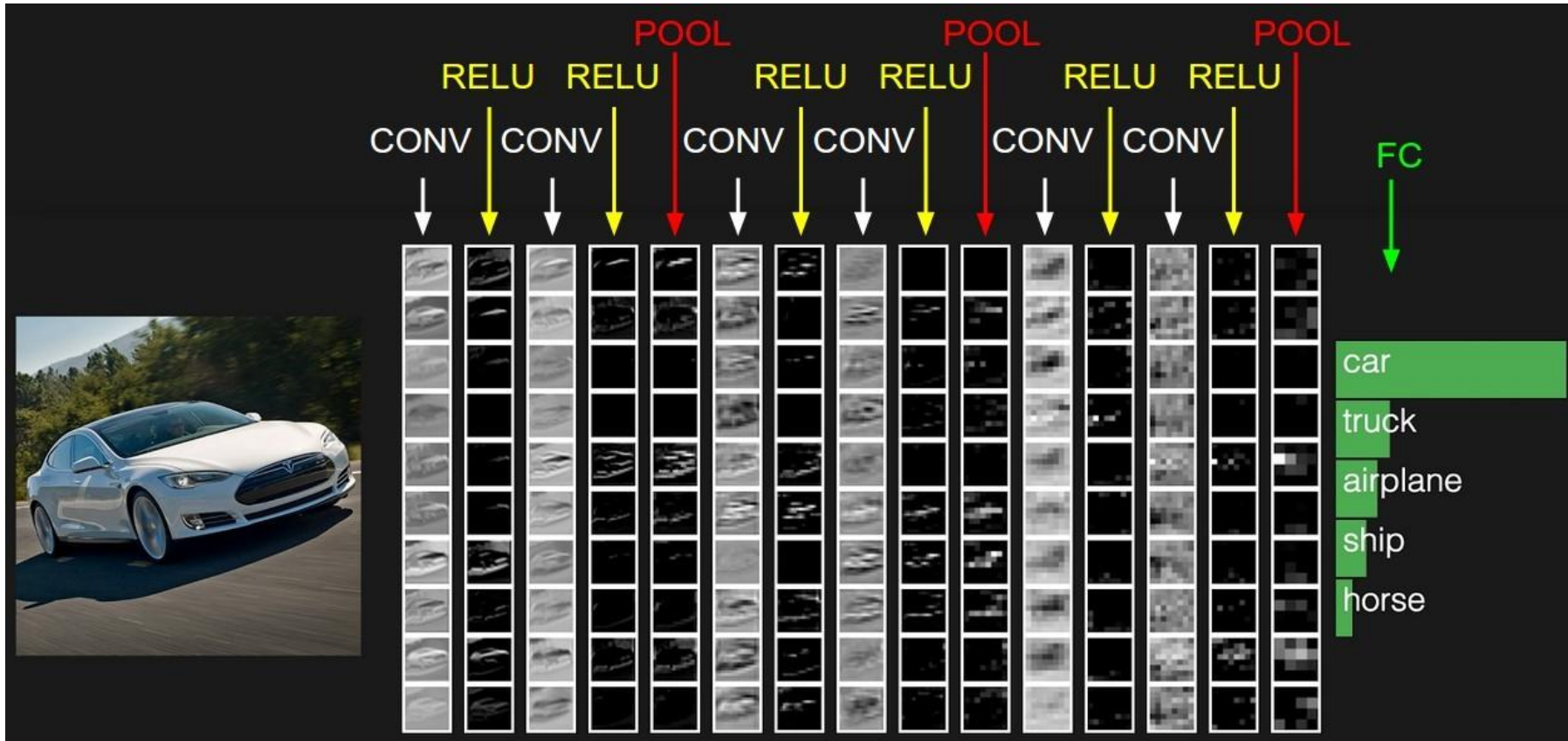


- JPEG 480 x 480
- Input for neural network: $480 \times 480 \times 3 = 691200$ numbers!

AI Fundamentals

Chapter 3: Neural Networks

Architecture of Convolutional Neural Networks (CNN)



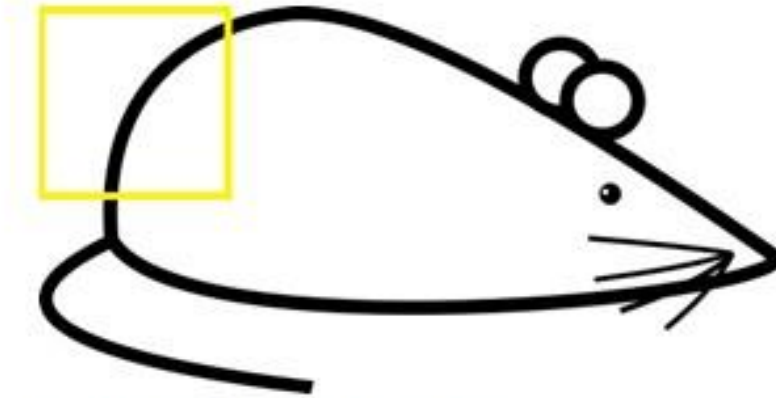
AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): detection of a 2D feature



Original image



Visualization of the filter on the image

URL: [2D convolution - confirmation of the presence of a feature](#)

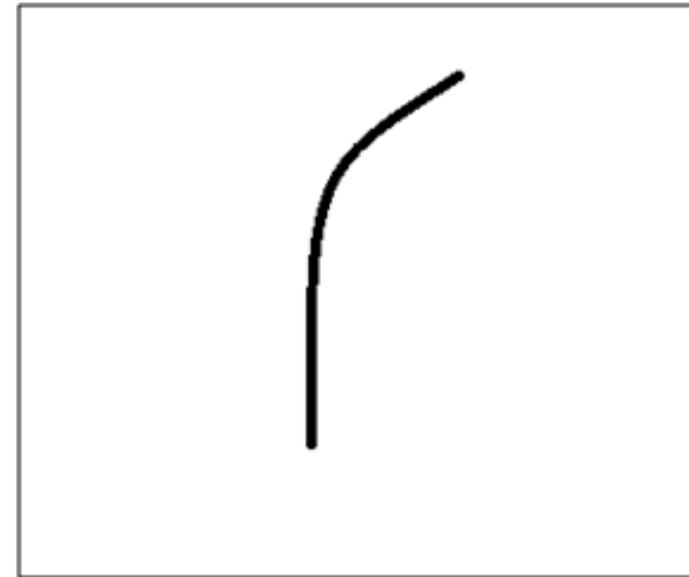
AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): detection of a 2D feature

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

URL: [2D convolution - confirmation of the presence of a feature](#)

AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): detection of a 2D feature



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = $(50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600$ (A large number!)

URL: [2D convolution - confirmation of the presence of a feature](#)

AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): detection of a 2D feature



Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = 0

URL: [2D convolution - absence of a feature](#)

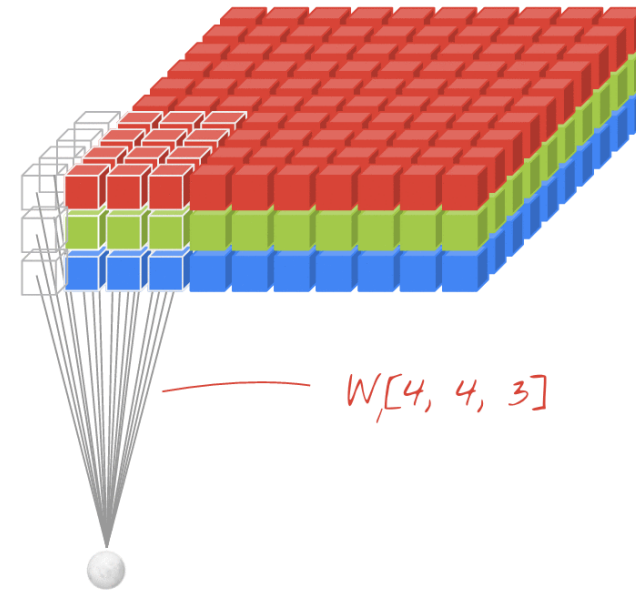
AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): 2D convolution

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		



URL: [2D convolution](#)

AI

Ch

AI

Ch

AI

Ch

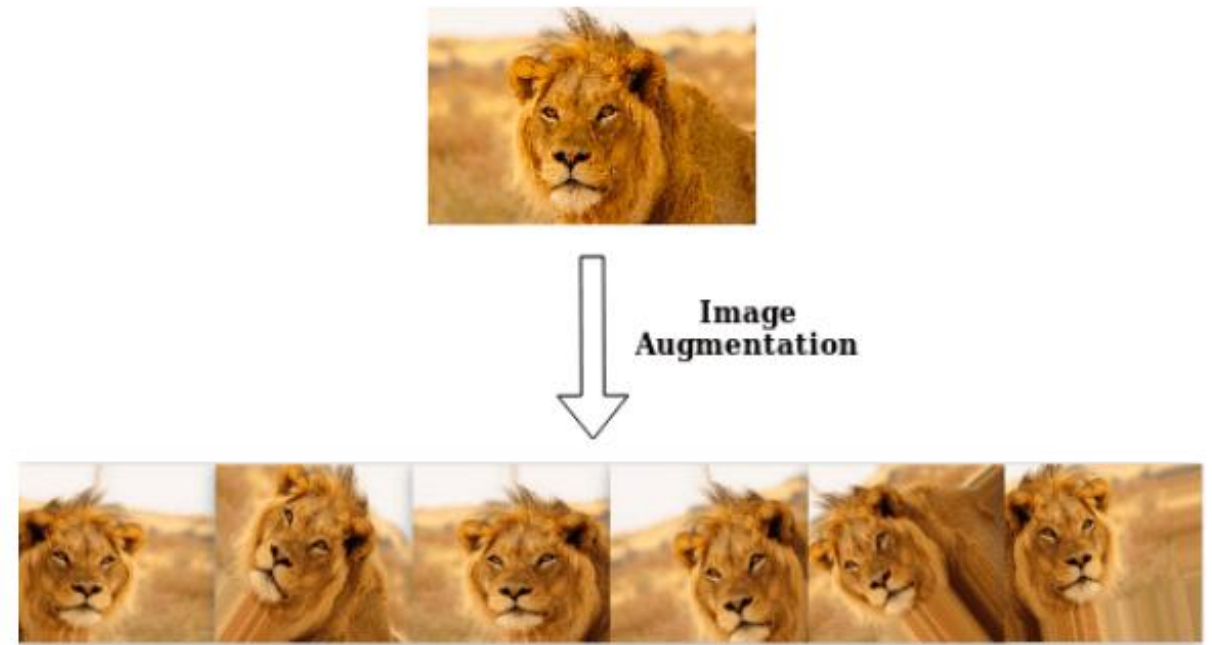
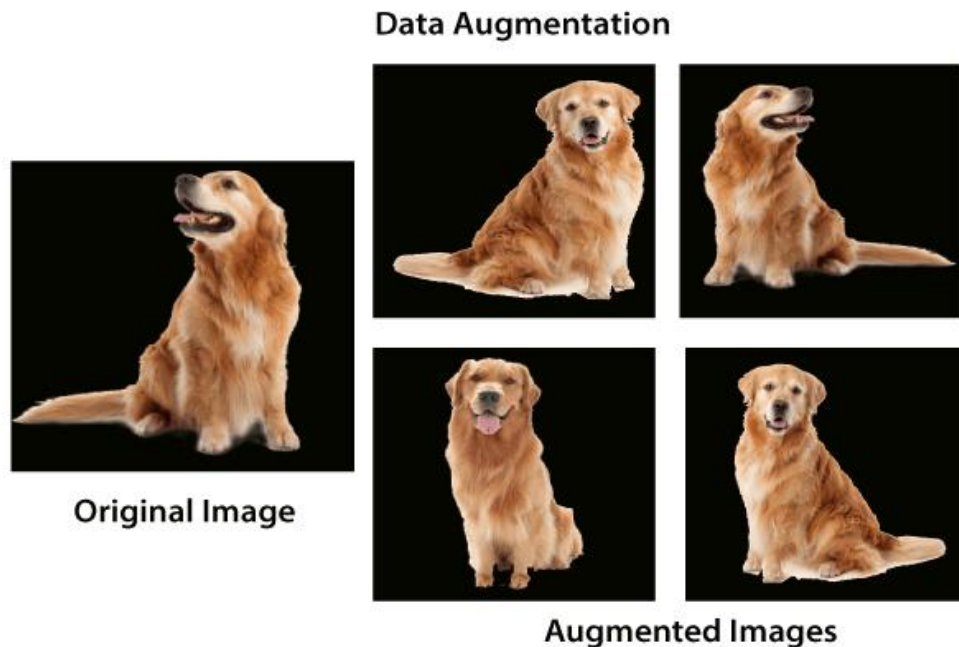


AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): augmented images

- Be careful of **overfitting**
- Data augmentation!



URL: [Data Augmentation](#)

URL: [Data Augmentation](#)

AI Fundamentals

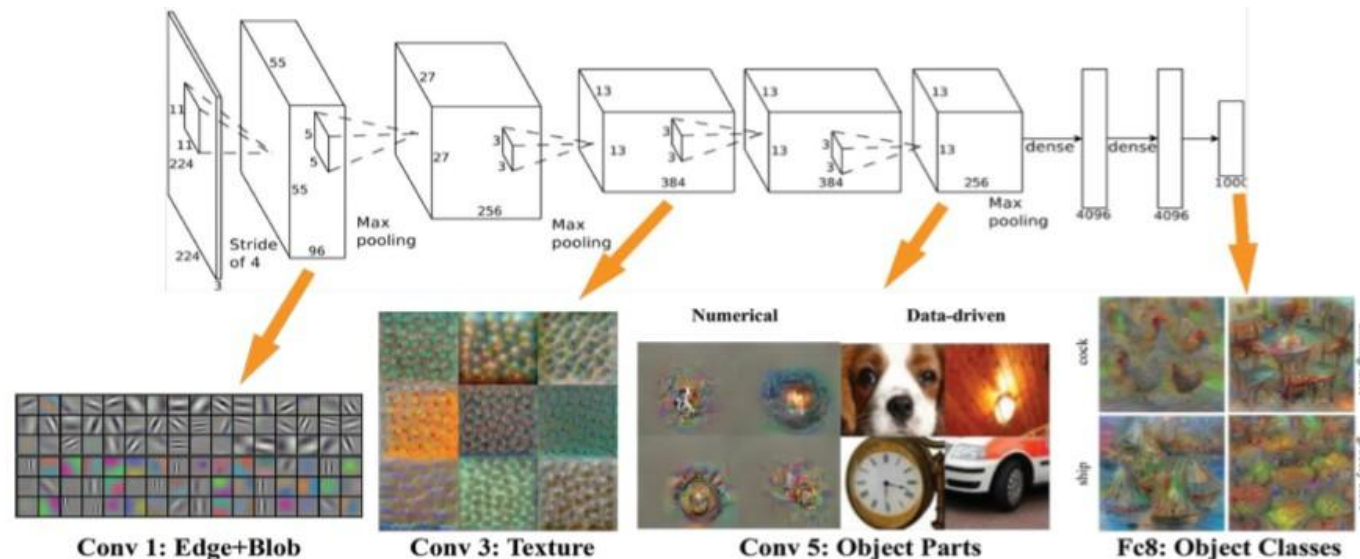
Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): layers

- **Dense layers** learn **global** patterns = look at entire picture
- **Convolutional layers** look at **local** patterns
- Combining convolutional layers = searching for combinations of patterns
- Key characteristics:
 - Patterns are translation invariant
 - They can learn spatial hierarchies of patterns

Convolutional Neural Network (CNN): Convolution Layers & Pooling Layers

- A **convolution layer** is a **feature detector** that automatically learns to filter out not needed information from an input by using convolution kernel.
- **Pooling layers** compute the **max** or **average** value of a particular feature over a region of the input data (downsizing of input images). Also helps to detect objects in some unusual places and reduces memory size.



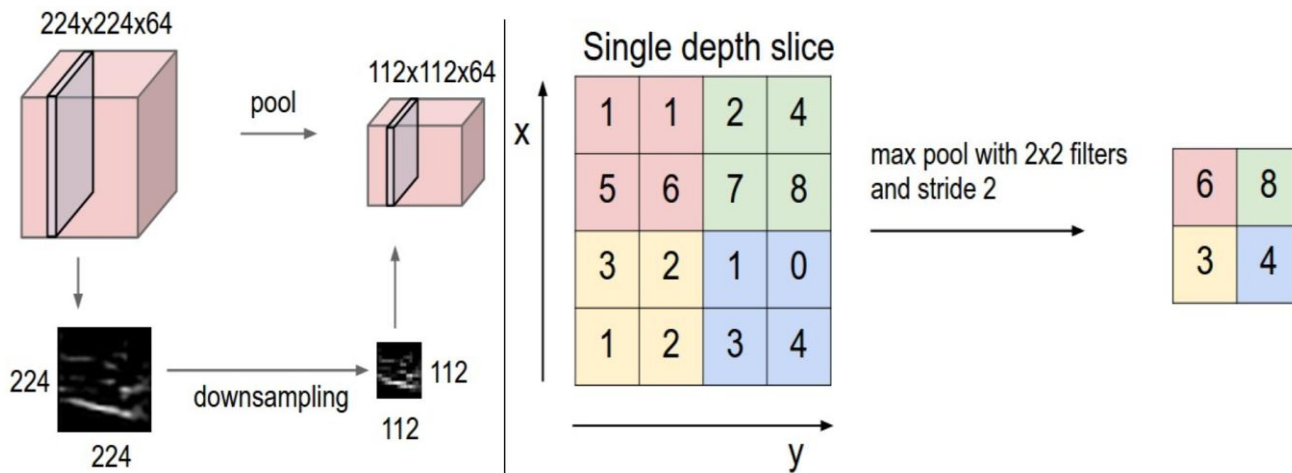
URL: [Visualization obtained at each layer](#)

AI Fundamentals

Chapter 3: Neural Networks

Convolutional Neural Networks (CNN): Pooling layers

- Pooling layers** compute the **max** or **average** value of a particular feature over a region of the input data (downsizing of input images). Also helps to detect objects in some unusual places and reduces memory size.



Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2, stride 2 into output volume of size $[112 \times 112 \times 64]$. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2×2 square).

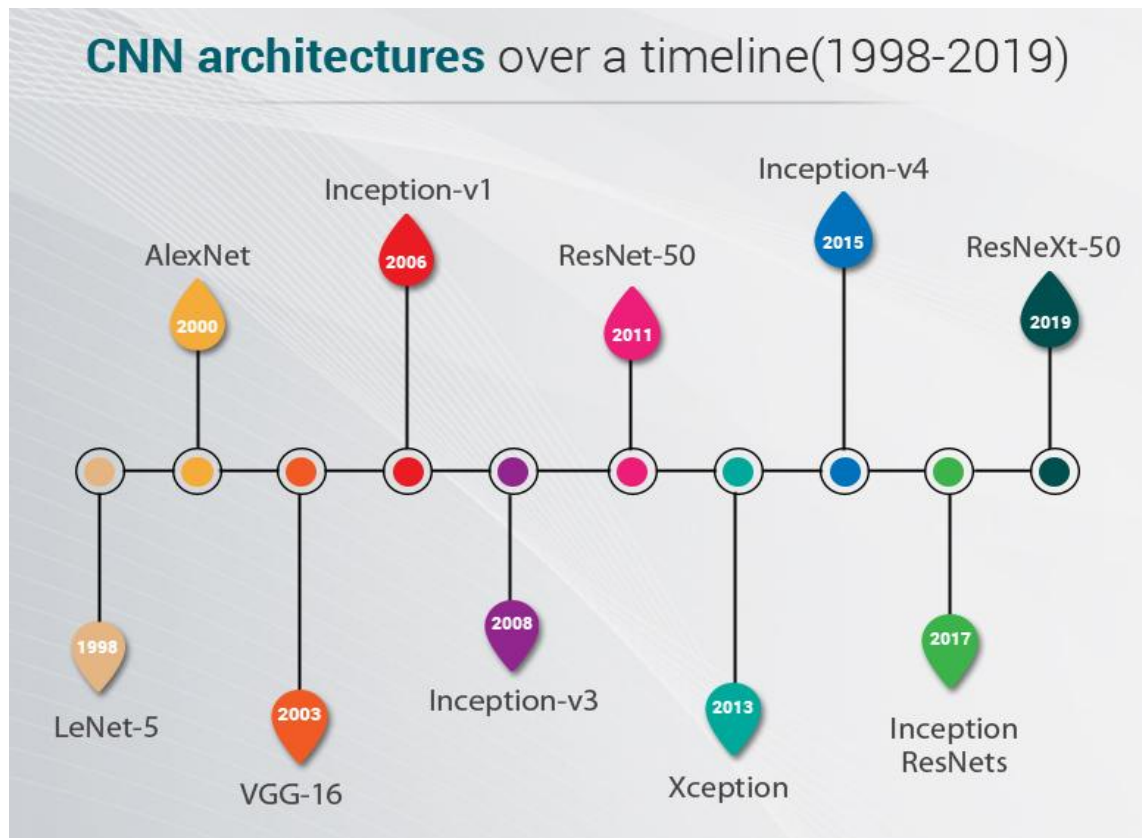
3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

AI Fundamentals

Chapter 3: Neural Networks

Deep Learning: Network Types - Convolutional Neural Networks (CNN)



- Some models that can be used:
 - Xception
 - Inception V3
 - ResNet50
 - VGG16
 - VGG19
 - MobileNet

URL: [CNN Architectures over a timeline](#)

AI Fundamentals

Chapter 3: Neural Networks

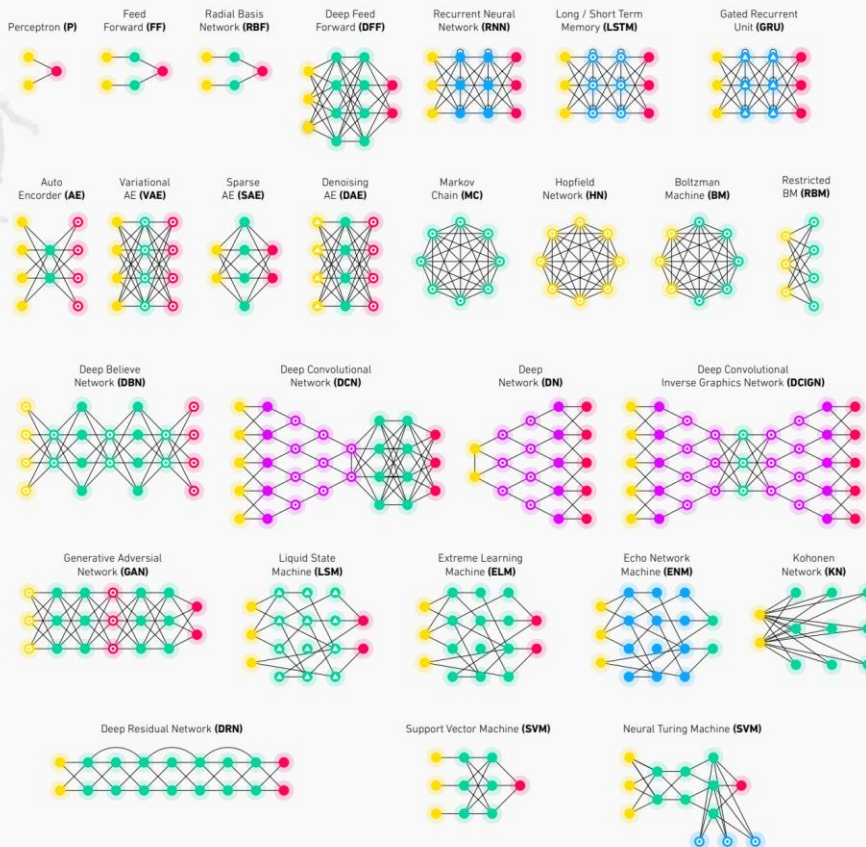
Deep Learning: Neural Network Types

Neural Networks Basic Cheat Sheet

BecomingHuman.AI

Index

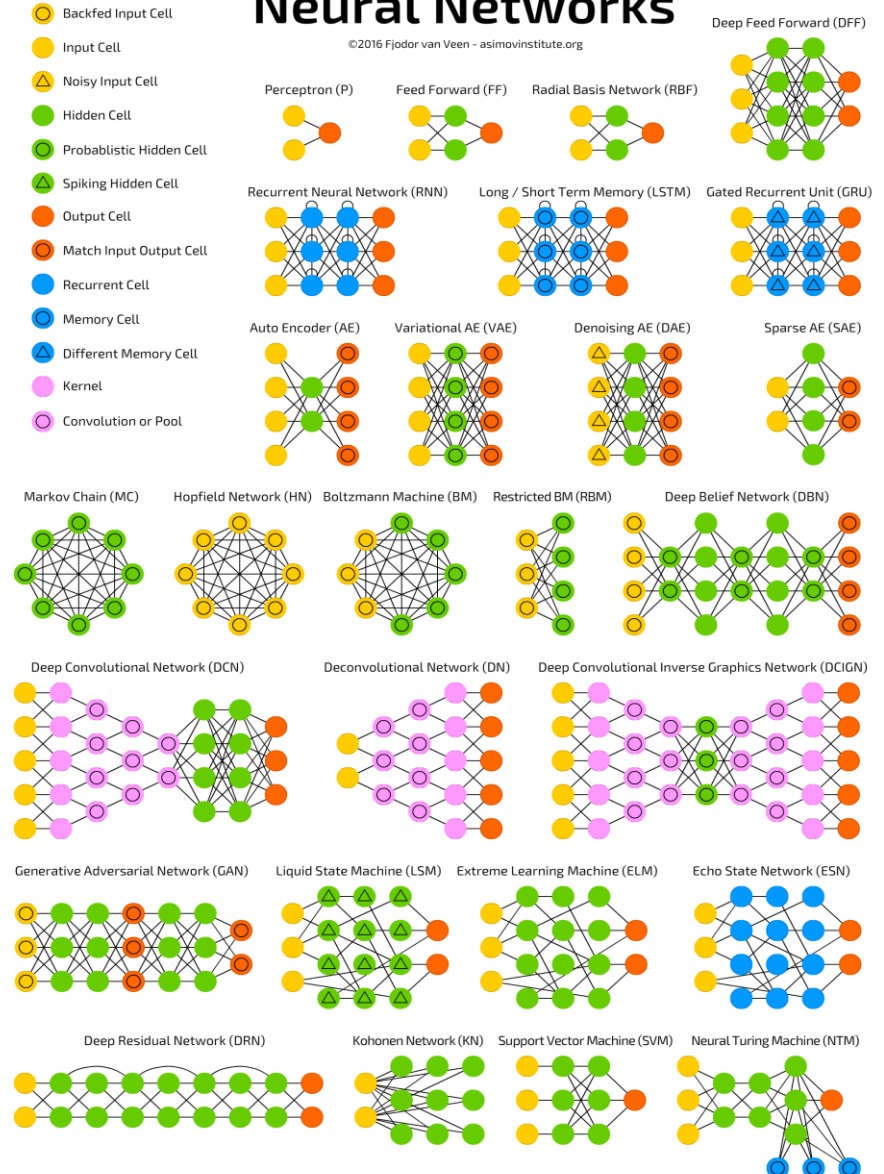
- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolutional or Pool



Original Copyright by AsimovInstitute.org See original here

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

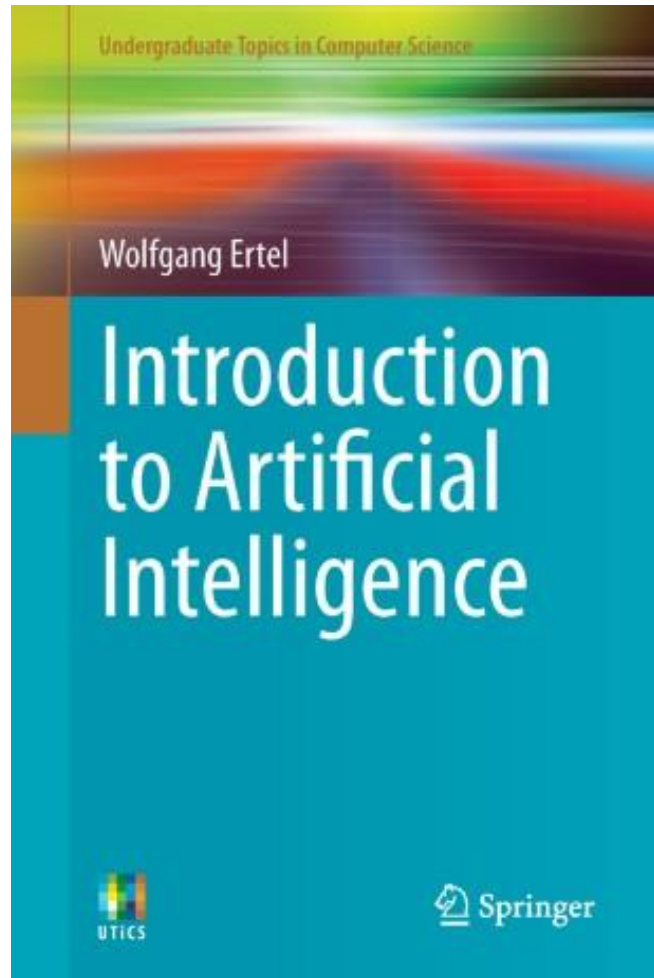


URL: [Neural Networks Basic - Cheat Sheet](#)

URL: [A mostly complete chart of neural networks](#)

AI Fundamentals

End of Chapter 3 – Neural Networks



VIVES University of Applied Sciences
Bachelor in electronics-ICT

